# DSCI-560
## Lecture 12: Classification and Clustering

Data Science Professional Practicum

Young Cho
Department of Electrical Engineering
University of Southern California

# Learning Approaches

## Supervised Learning

- The **training data** is **annotated** with information to help the learning system
  - Eg. the class for each instance

## Unsupervised Learning

- The training data is **not annotated** with any extra information to help the learning system
  - Eg. clustering of data

## Semi-Supervised Learning

# Different Data Analysis Tasks

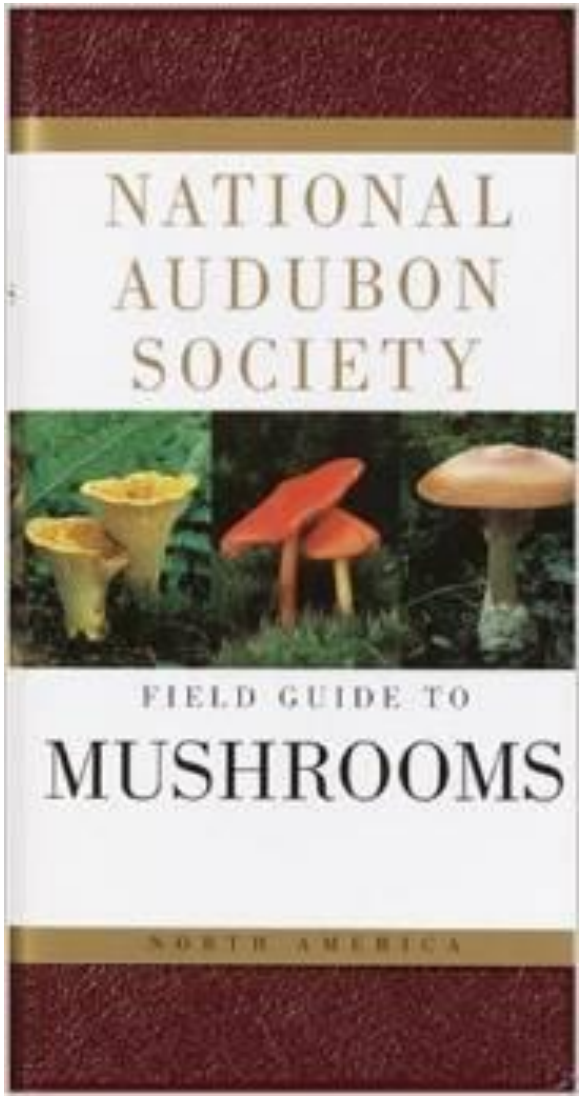
- **Classification**
  - Assign a category (ie, a class) for a new instance
- **Clustering**
  - Form clusters (ie, groups) with a set of instances
- **Pattern detection**
  - Identify regularities (ie, patterns) in temporal or spatial data
- **Simulation**
  - Define mathematical formulas that can generate data similar to observations collected

# Different Data Analysis Tasks

- **Classification**
- **Clustering**
- **Pattern detection**
- **Causal discovery**
- **Simulation**
- **…**

- Each **type of task** is characterized by the **kinds of data** they require and the **kinds of output** they generate
- Each type of task uses different algorithms.

# Classifying Mushrooms




- What mushrooms are edible, i.e., not poisonous?
  - Book **lists many kinds of mushrooms** identified as either edible, poisonous, or unknown edibility
  - Given a **new kind** mushroom not listed in the book, is it edible?

https://archive.ics.uci.edu/ml/datasets/Mushroom

# Classifying Iris Plants

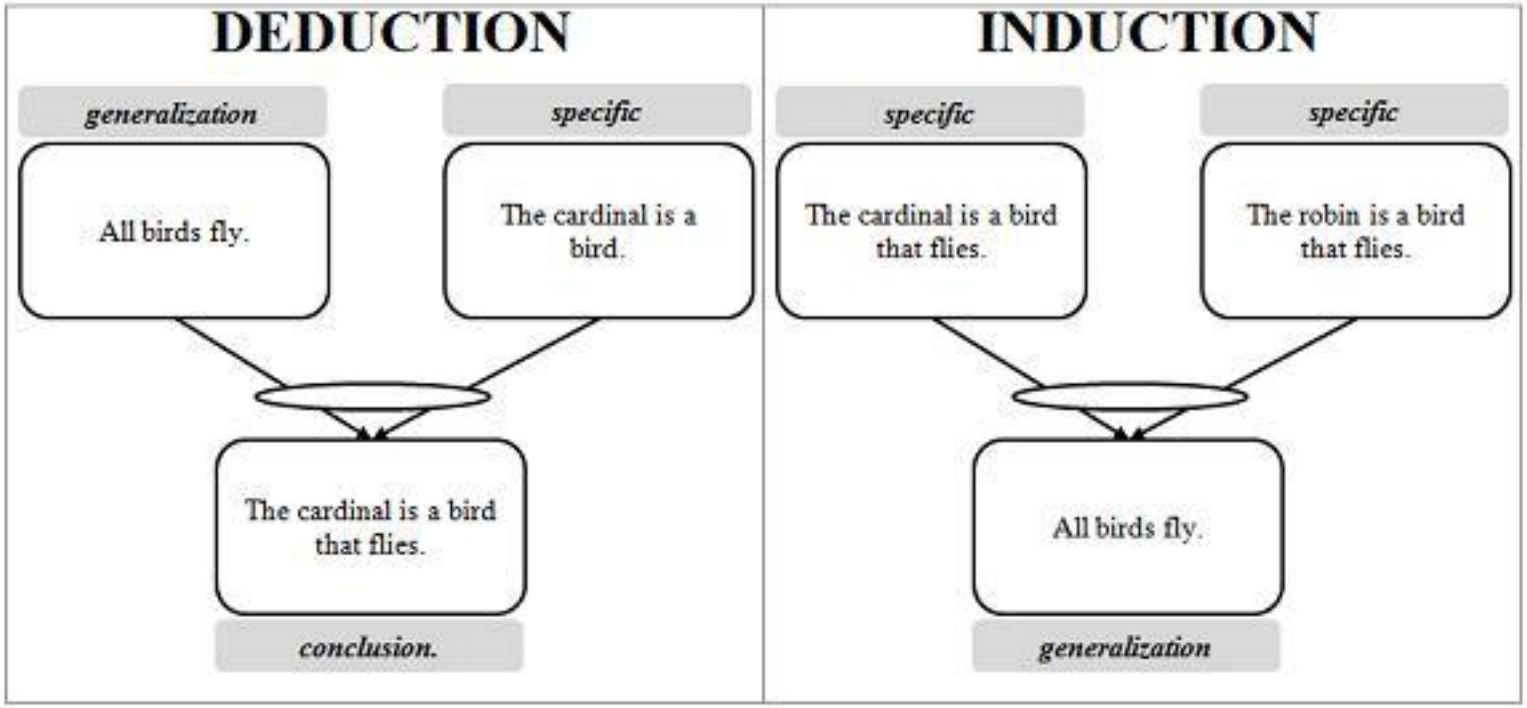
- Iris flowers have different sepal and petal shapes:
  - Iris Setosa
  - Iris Versicolour
  - Iris Virginica

- Suppose you are shown lots of examples of each type.
- Given a **new iris** flower, what type is it?

htt

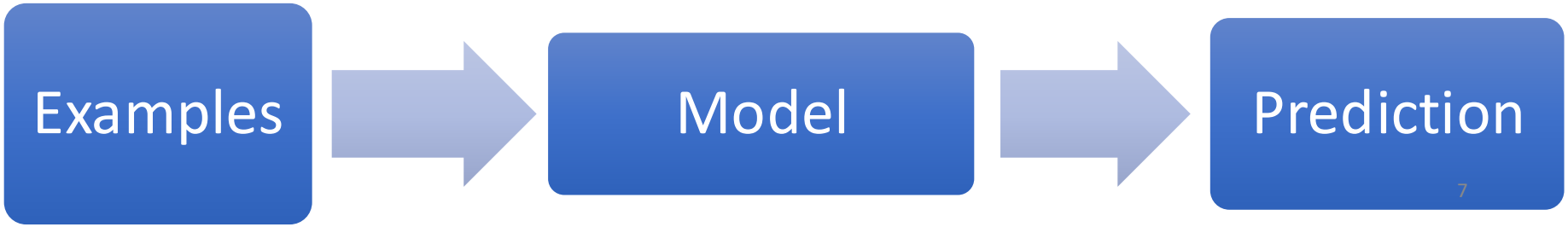
# Induction

Classifiers use induction: they generate general rules about the target classes



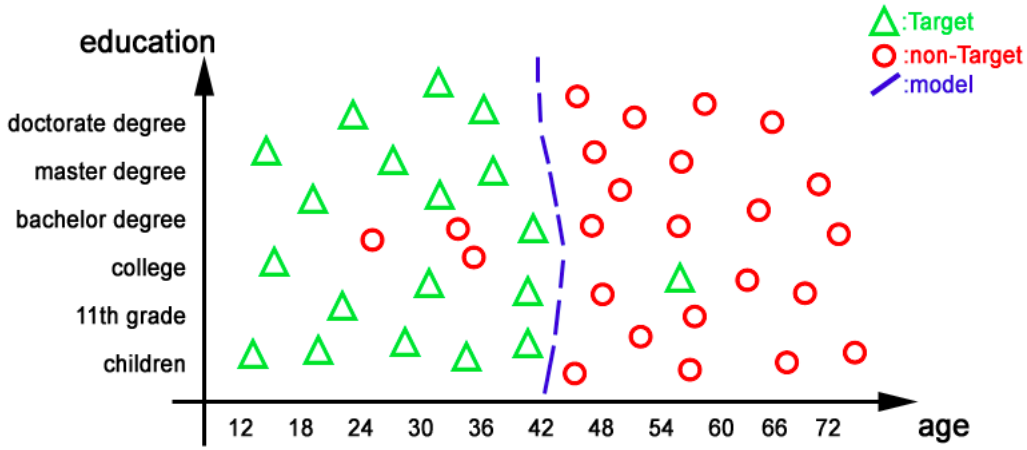

Generalize

Instantiate for another case



# Classification Tasks

- Given:
  - A set of classes
  - Instances (examples) of each class
- Generate: A method (aka *model*) that when given a new instance it will determine its class.






http://www.business-insight.com/html/intelligence/bi_overfitting.html

# Classification Tasks

- Given:
  - A set of classes
  - Instances of each class
- Generate: A method that when given a new instance it will determine its class

- Instances are described as a set of *features* or *attributes* and their *values*
- The class that the instance belongs to is also called its "*label*"
  - Input is a set of "labeled instances".

# Decisions for Classification Task

- What features to choose
  - Try defining different **features**
  - For some problems, hundreds and maybe thousands of features may be possible
  - Sometimes the features are **not directly** observable (ie, there are "*latent*" variables)

- What classes to choose
  - Edible / poisonous?
  - Edible / poisonous / unknown?
- How many labeled examples
  - May require a lot of work
- What modeler to choose
  - Better to try different ones

# Possible Features

1. cap-shape: bell=b,conical=c,convex=x,flat=f, knobbed=k,sunken=s
2. cap-surface: fibrous=f,grooves=g,scaly=y,smooth=s
3. cap-color: brown=n,buff=b,cinnamon=c,gray=g,green=r, pink=p,purple=u,red=e,white=w,yellow=y
4. bruises?: bruises=t,no=f
5. odor: almond=a,anise=l,creosote=c,fishy=y,foul=f, musty=m,none=n,pungent=p,spicy=s
6. gill-attachment: attached=a,descending=d,free=f,notched=n
7. gill-spacing: close=c,crowded=w,distant=d
8. gill-size: broad=b,narrow=n
9. gill-color: black=k,brown=n,buff=b,chocolate=h,gray=g, green=r,orange=o,pink=p,purple=u,red=e, white=w,yellow=y
10. stalk-shape: enlarging=e,tapering=t
11. stalk-root: bulbous=b,club=c,cup=u,equal=e, rhizomorphs=z,rooted=r,missing=?
12. stalk-surface-above-ring: fibrous=f,scaly=y,silky=k,smooth=s
13. stalk-surface-below-ring: fibrous=f,scaly=y,silky=k,smooth=s
14. stalk-color-above-ring: brown=n,buff=b,cinnamon=c,gray=g,orange=o, pink=p,red=e,white=w,yellow=y
15. stalk-color-below-ring: brown=n,buff=b,cinnamon=c,gray=g,orange=o, pink=p,red=e,white=w,yellow=y
16. veil-type: partial=p,universal=u
17. veil-color: brown=n,orange=o,white=w,yellow=y
18. ring-number: none=n,one=o,two=t
19. ring-type: cobwebby=c,evanescent=e,flaring=f,large=l, none=n,pendant=p,sheathing=s,zone=z
20. spore-print-color: black=k,brown=n,buff=b,chocolate=h,green=r, orange=o,purple=u,white=w,yellow=y
21. population: abundant=a,clustered=c,numerous=n, scattered=s,several=v,solitary=y
22. habitat: grasses=g,leaves=l,meadows=m,paths=p, urban=u,waste=w,woods=d



https://commons.wikimedia.org/wiki/File:Twelve_edible_mushrooms_of_the_United_States.jpg

# Describing an Instance

p,x,s,n,t,p,f,c,n,k,e,e,s,s,w,w,p,w,o,p,k,s,u

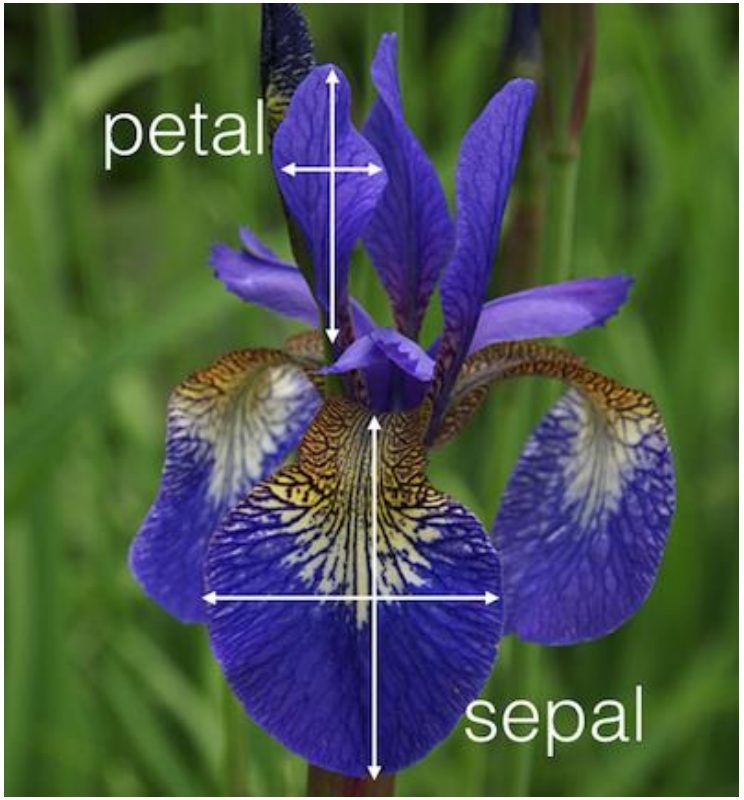
- Class: poisonous - p
- Cap shape: convex – x
- Cap surface: smooth – s
- Cap color: brown – n
- Bruises: true – t
- Odor: pungent – p
- …

https://en.wikipedia.org/wiki/Edible_mushroom#/media/File:Lepista_nuda.jpg

# Describing an Instance- Iris

- Attribute Information (All in centimeters)
  - Sepal length
  - Sepal width
  - Petal length
  - Petal width
  - Flower class

Ex: 5.3,3.7,1.5,0.2,Iris-setosa
5.0,3.3,1.4,0.2,Iris-setosa
7.0,3.2,4.7,1.4,Iris-versicolor
6.4,3.2,4.5,1.5,Iris-versicolor
6.3,3.3,6.0,2.5,Iris-virginica
5.8,2.7,5.1,1.9,Iris-virginica



# Iris Classification: "*Continuous*" Feature Values

1. sepal length in cm
2. sepal width in cm
3. petal length in cm
4. petal width in cm
5. class:

-- Iris Setosa
-- Iris Versicolour
-- Iris Virginica

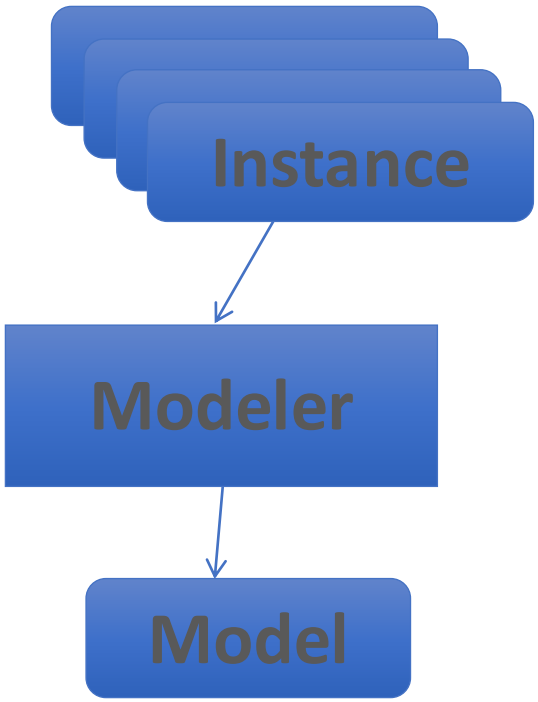
# Describing Many Instances

p,x,s,n,t,p,f,c,n,k,e,e,s,s,w,w,p,w,o,p,k,s,u
e,x,s,y,t,a,f,c,b,k,e,c,s,s,w,w,p,w,o,p,n,n,g
e,b,s,w,t,l,f,c,b,n,e,c,s,s,w,w,p,w,o,p,n,n,m
p,x,y,w,t,p,f,c,n,n,e,e,s,s,w,w,p,w,o,p,k,s,u
e,x,s,g,f,n,f,w,b,k,t,e,s,s,w,w,p,w,o,e,n,a,g
e,x,y,y,t,a,f,c,b,n,e,c,s,s,w,w,p,w,o,p,k,n,g
e,b,s,w,t,a,f,c,b,g,e,c,s,s,w,w,p,w,o,p,k,n,m
e,b,y,w,t,l,f,c,b,n,e,c,s,s,w,w,p,w,o,p,n,s,m
p,x,y,w,t,p,f,c,n,p,e,e,s,s,w,w,p,w,o,p,k,v,g
e,b,s,y,t,a,f,c,b,g,e,c,s,s,w,w,p,w,o,p,k,s,m
e,x,y,y,t,l,f,c,b,g,e,c,s,s,w,w,p,w,o,p,n,n,g
e,x,y,y,t,a,f,c,b,n,e,c,s,s,w,w,p,w,o,p,k,s,m

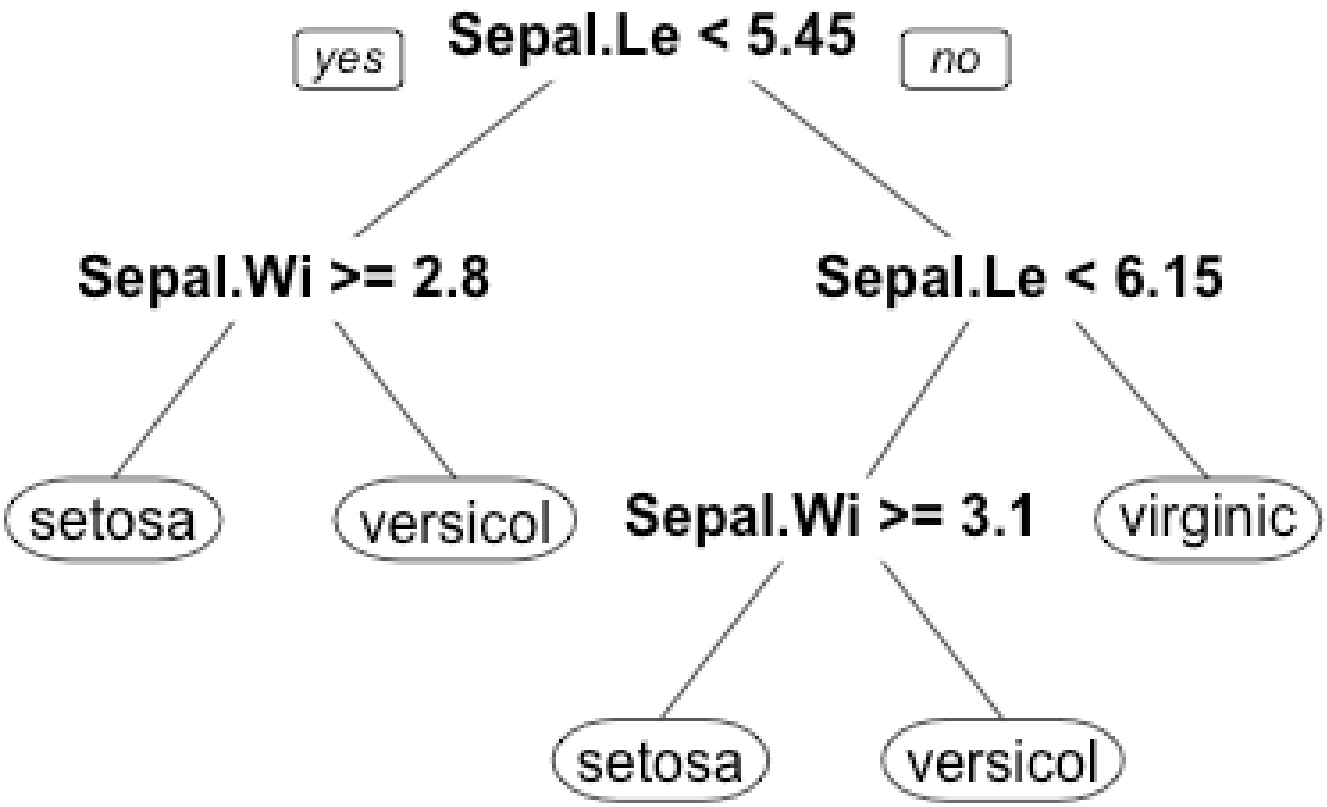
https://commons.wikimedia.org/wiki/File:Twelve_edible_mushrooms_of_the_United_States.jpg

# Classification Tasks

- Given: A set of labeled instances
- Generate: A method (aka *model*) that when given a new instance it will hypothesize its class



# Example of a Model:
# A Decision Tree

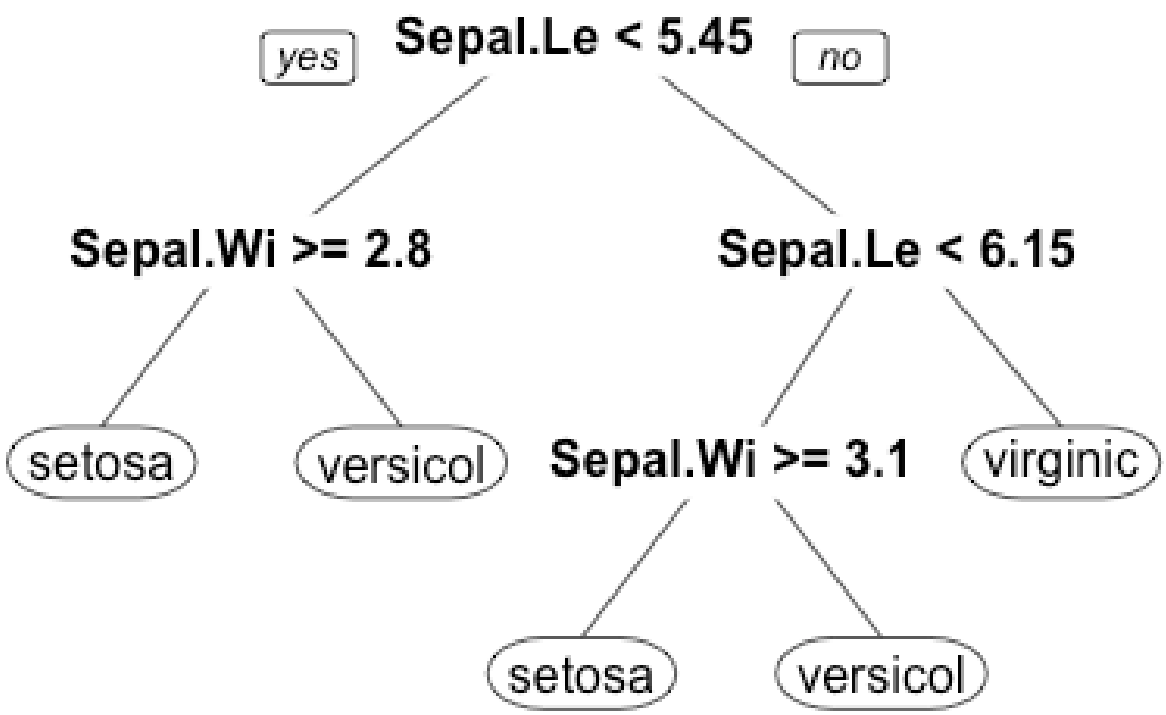



- **Nodes:** attribute-based decisions
- **Branches:** alternative values of the attributes
- **Leaves:** each leaf is a class



https://www.quora.com/What-are-the-disadvantages-of-using-a-decision-tree-for-classification

# Using a Decision Tree

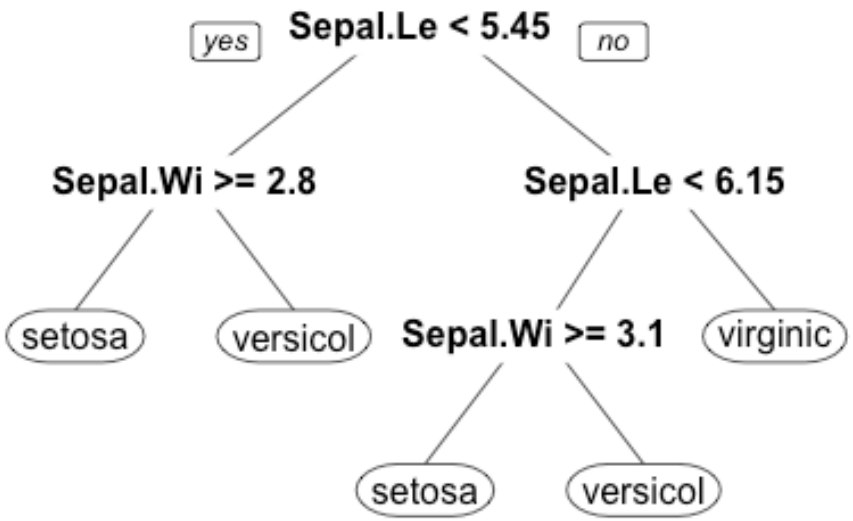



- Given a new instance, take a path through the tree based on its attributes
- When a leaf is reached, that is the class assigned to the instance.



https://www.quora.com/What-are-the-disadvantages-of-using-a-decision-tree-for-classification

# Learning Decision Tree



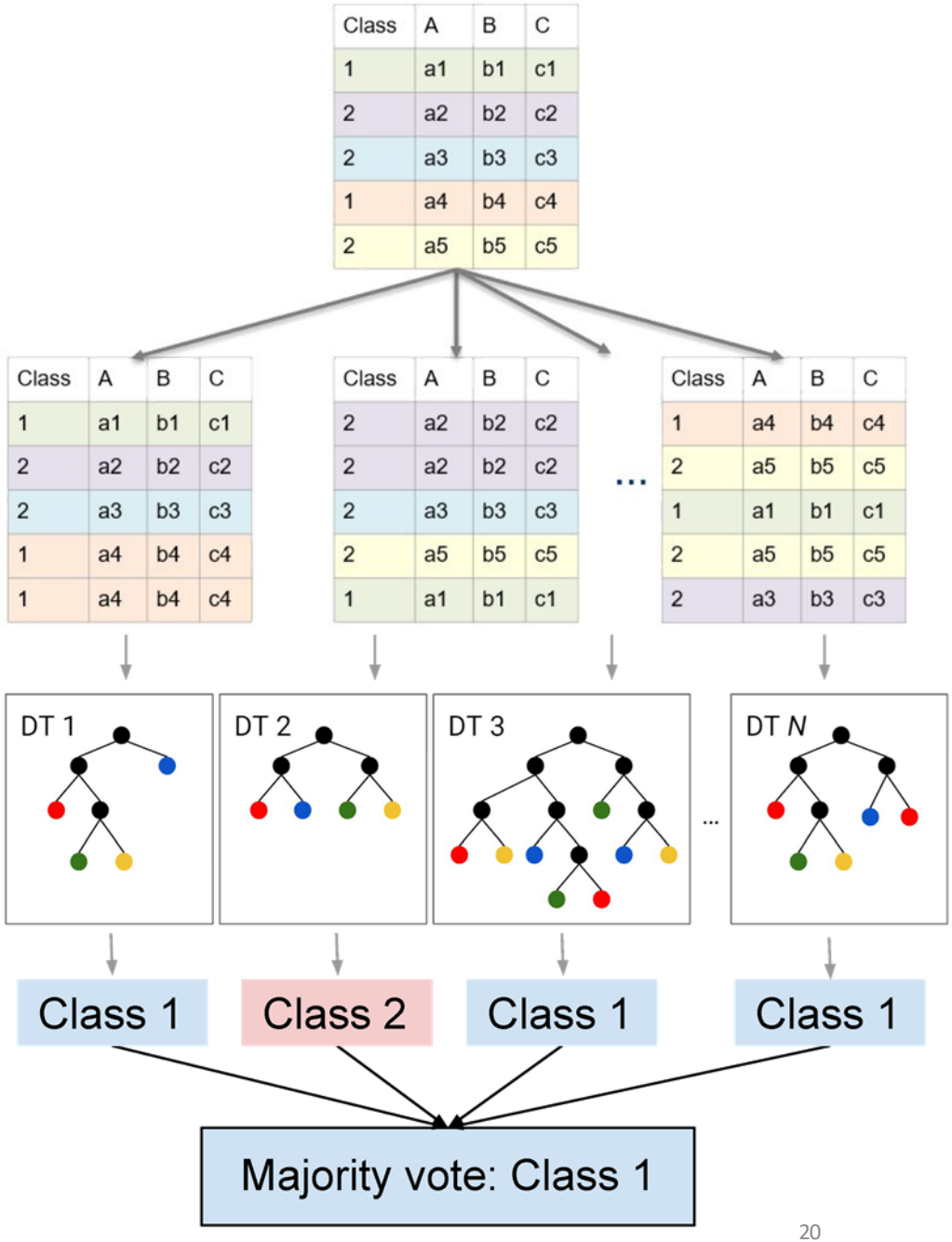

- Start with the set of all instances in the root node
- Select the attribute that splits the set best and create children nodes
  - Eg more evenly into the subsets
  - Use entropy to find maximum info
- When a node has all instances in the same class, make it a leaf node
- Iterate until all nodes are leaves.



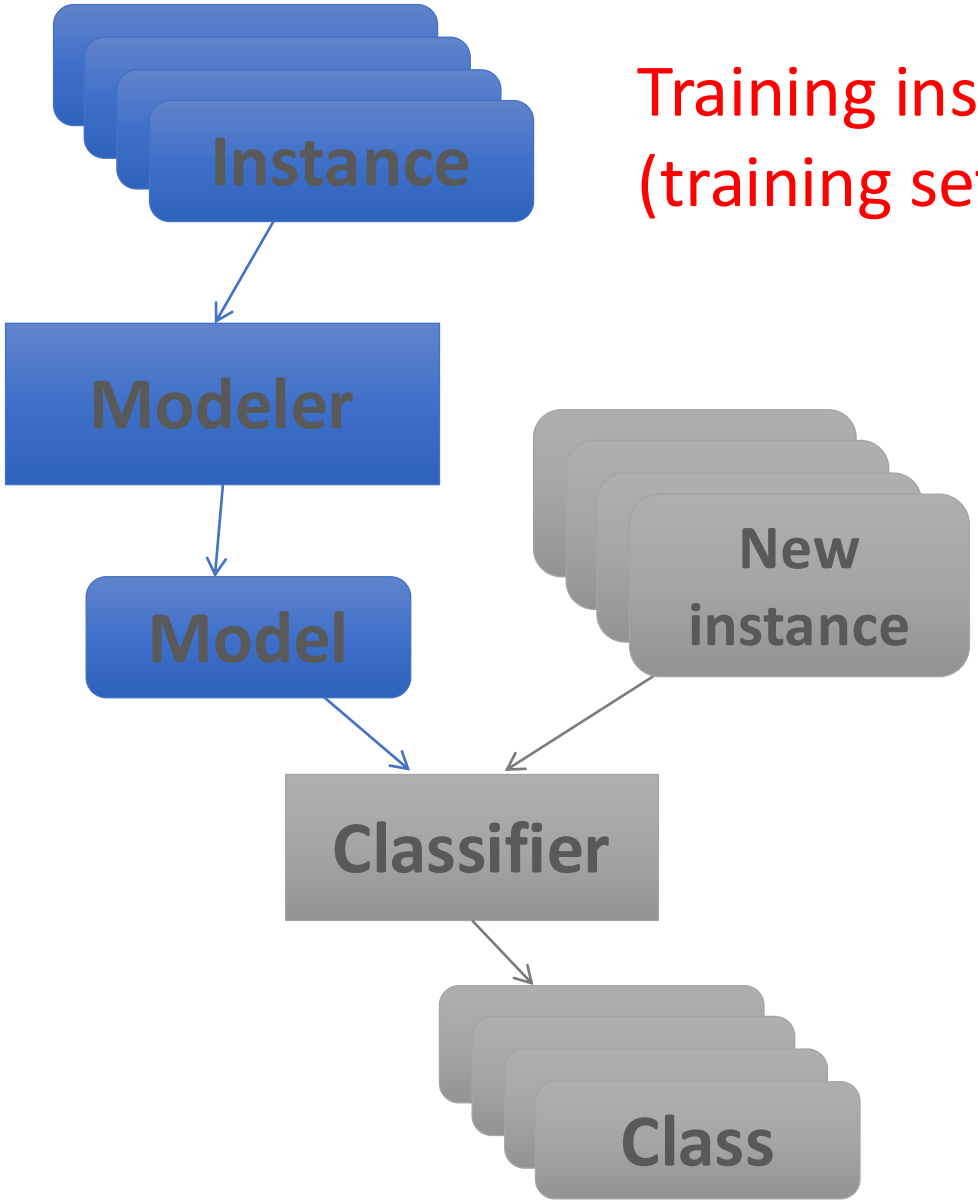
https://www.quora.com/What-are-the-disadvantages-of-using-a-decision-tree-for-classification

# Random Forest

- Bootstrap
  - Random Instances
  - Random Attributes
- Learn Decision Trees
  - Ensamble of Trees
- Classify
  - Majority Vote



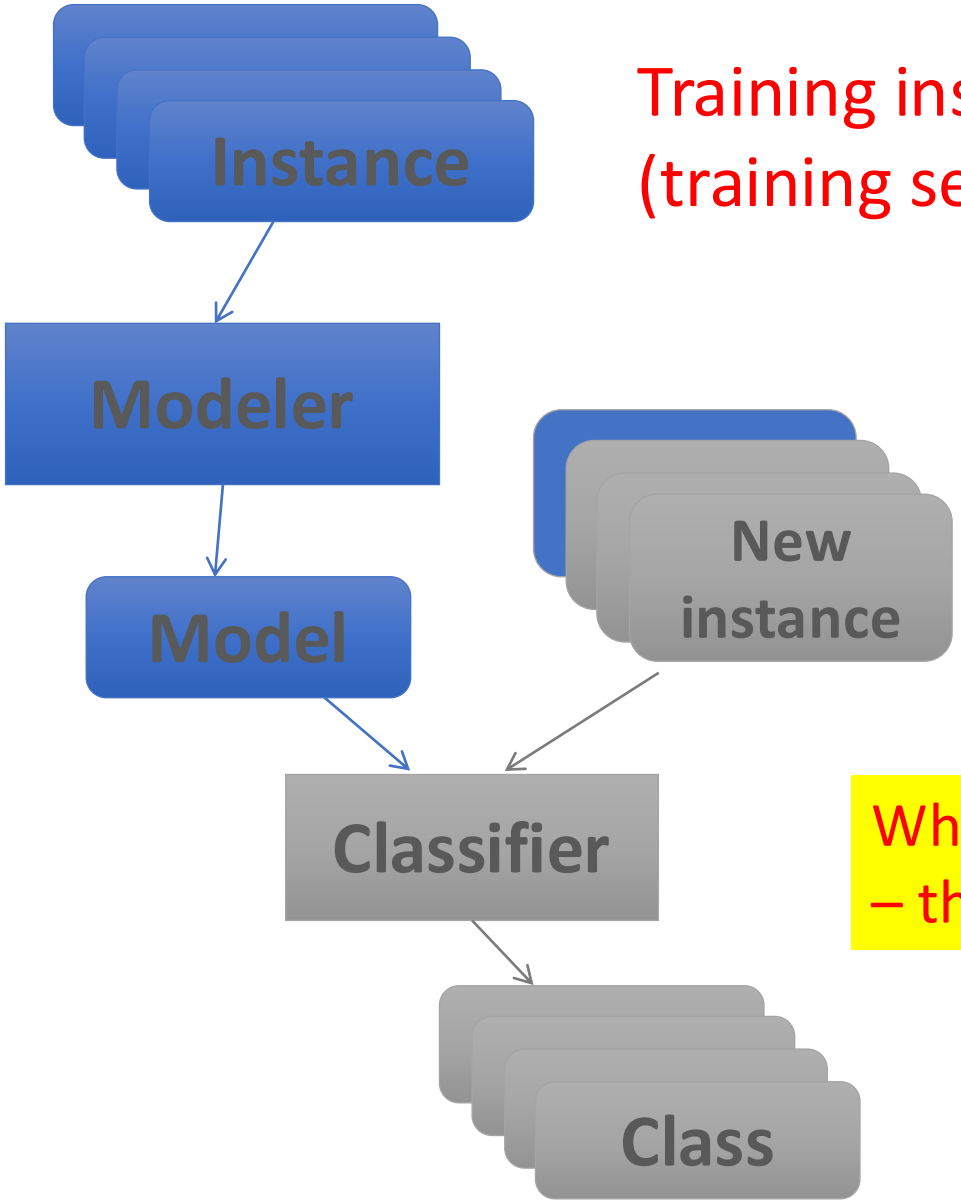
# Training and Test Sets




Training instances
(training set)

Test instances
(test set)

# Contamination




Training instances (training set)

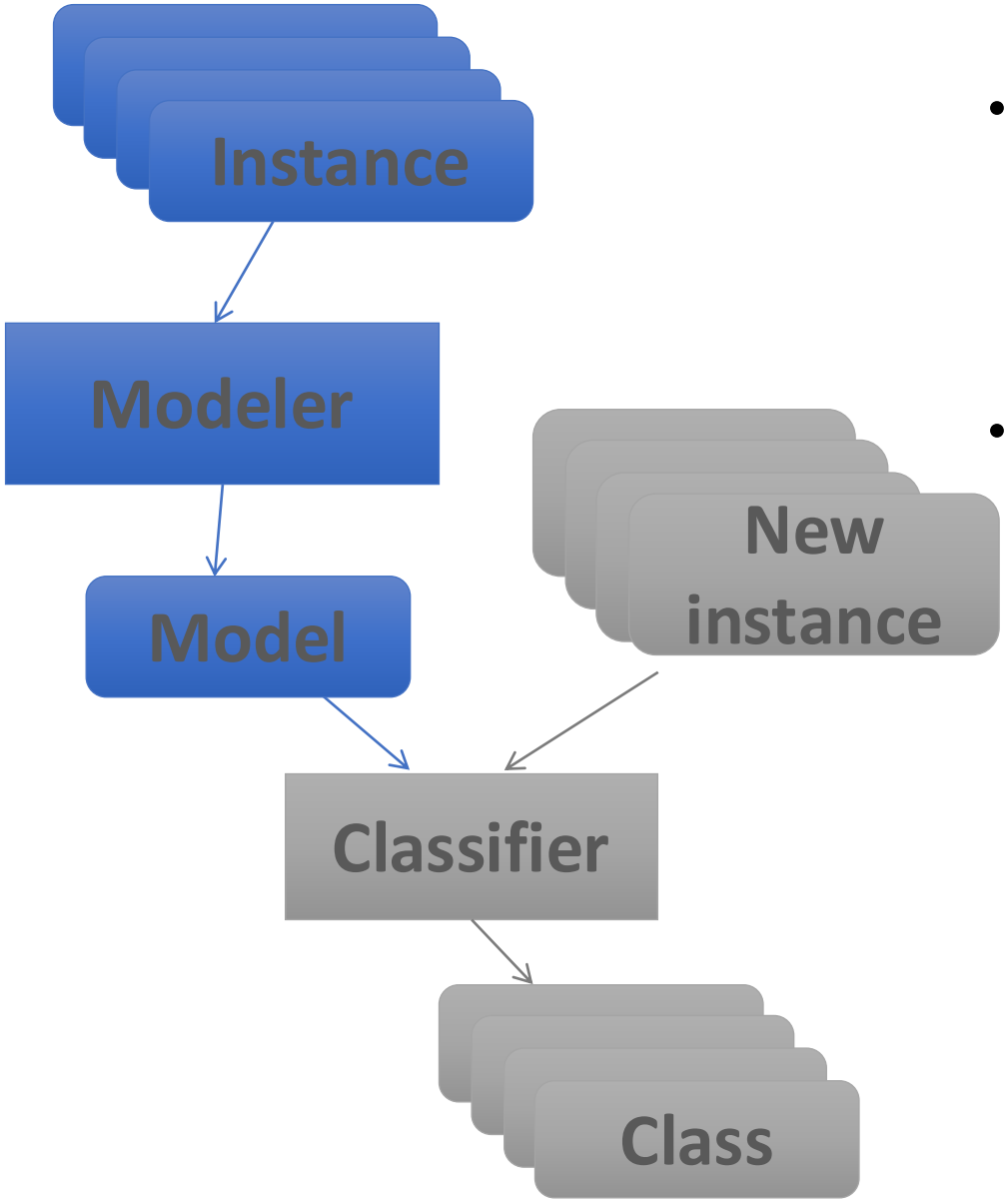
Test instances (test set)

When training and test sets overlap – this should NEVER happen

# About Classification Tasks

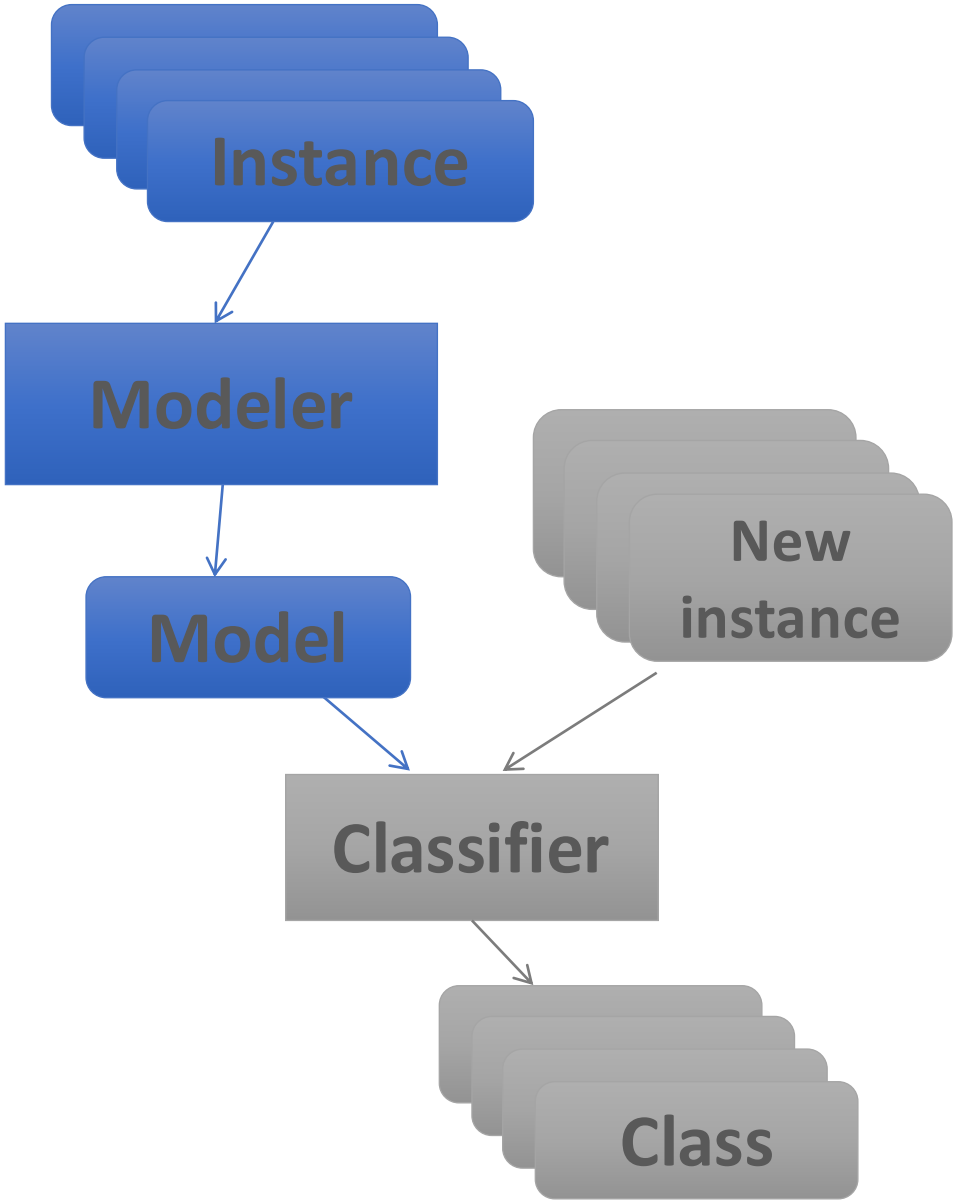
- Classes must be *disjoint*, ie, each instance belongs to only one class
- Classification tasks are "*binary*" if there are only two classes
- The classification method will rarely be perfect, it will make **mistakes** in its classification of new instances.

# What is a Modeler?

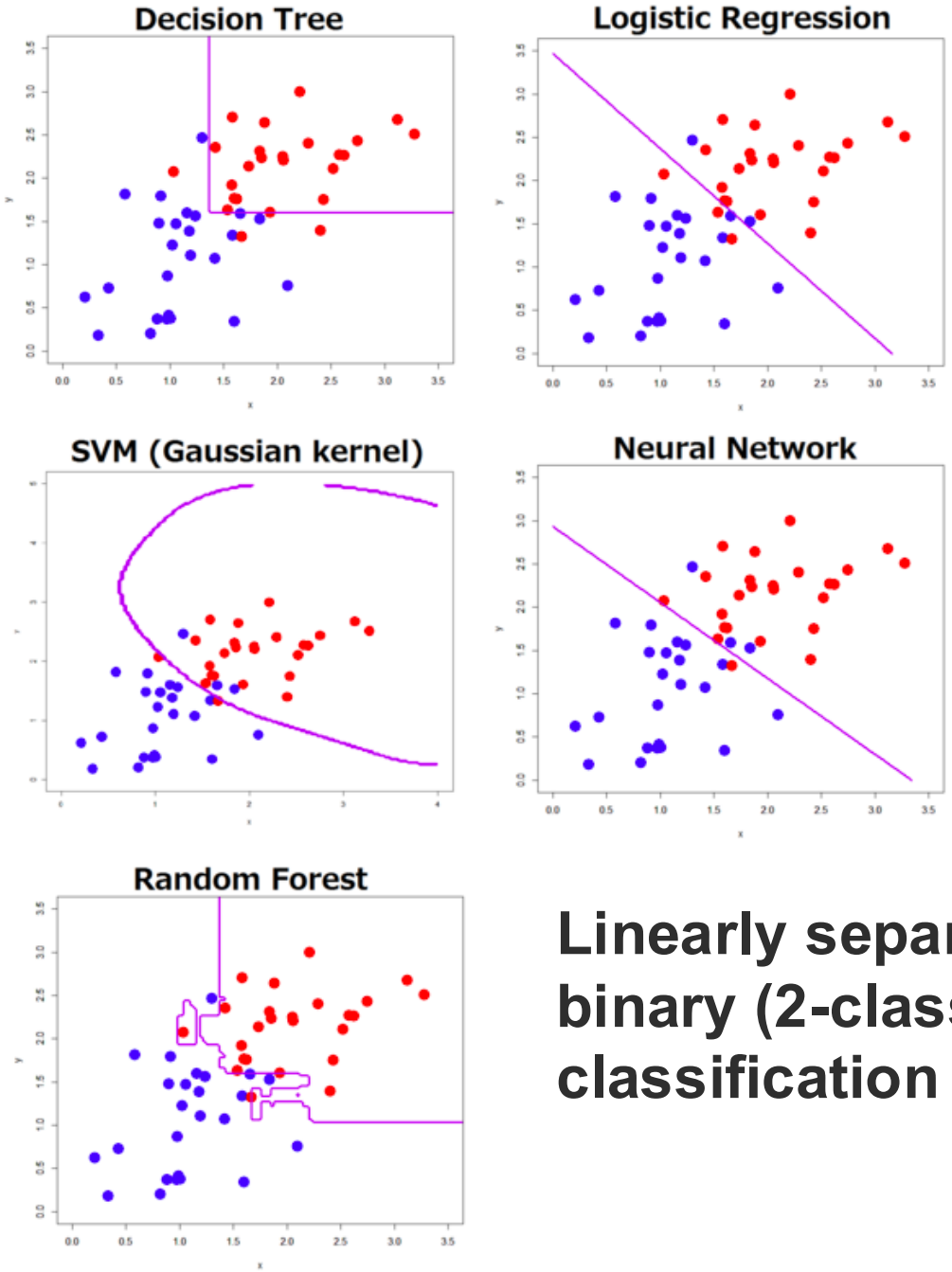



- A **mathematical / algorithmic** approach to generalize from instances so it can **make predictions** about instances that it has not seen before
- Its output is called a model.

# Types of Modelers/Models



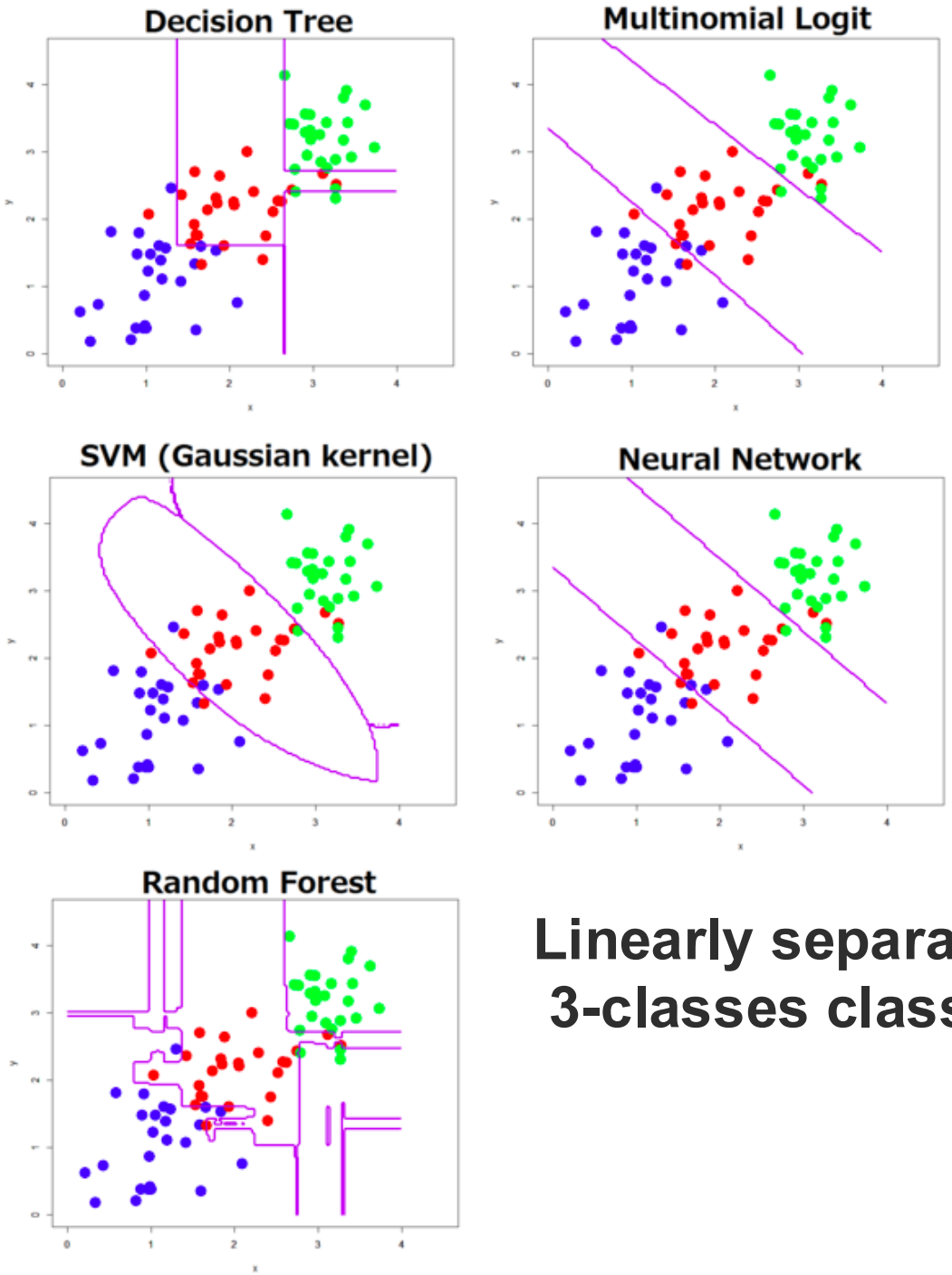

- Logistic regression
- Naïve Bayes classifiers
- Support vector machines (SVMs)
- Decision trees
- Random forests
- Kernel methods
- Genetic algorithms
- Neural networks




**Linearly separable pattern: binary (2-classes) classification**



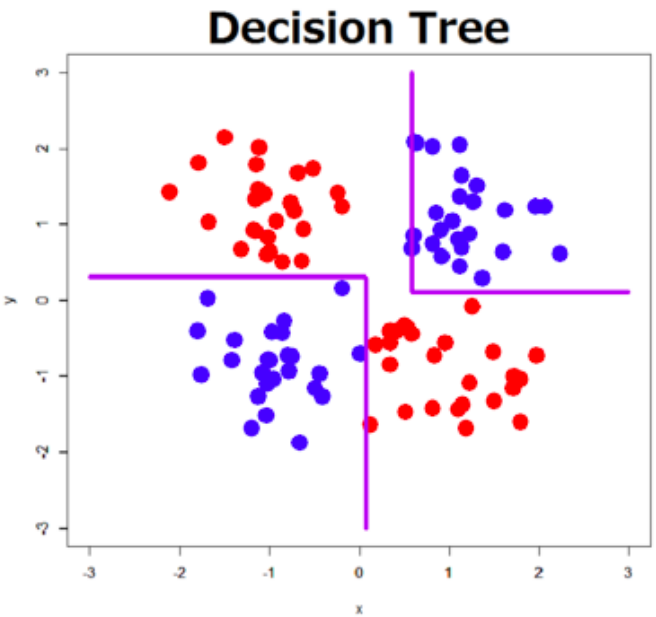
http://tjo-en.hatenablog.com/entry/2014/01/06/234155

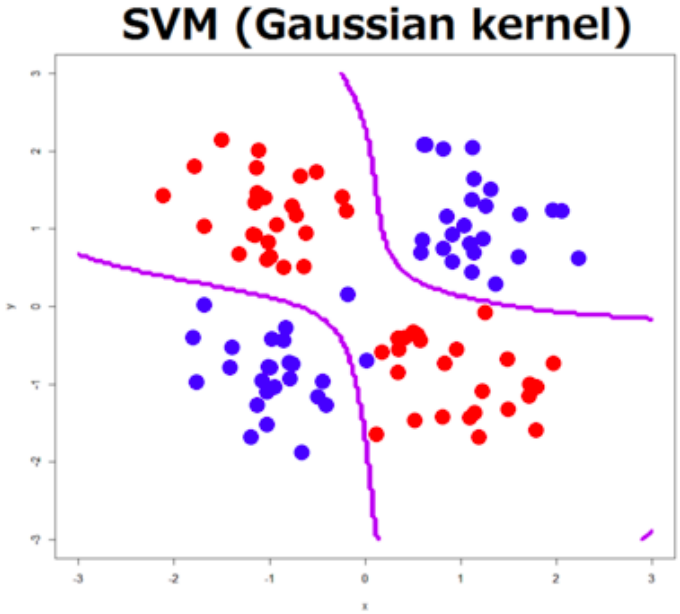


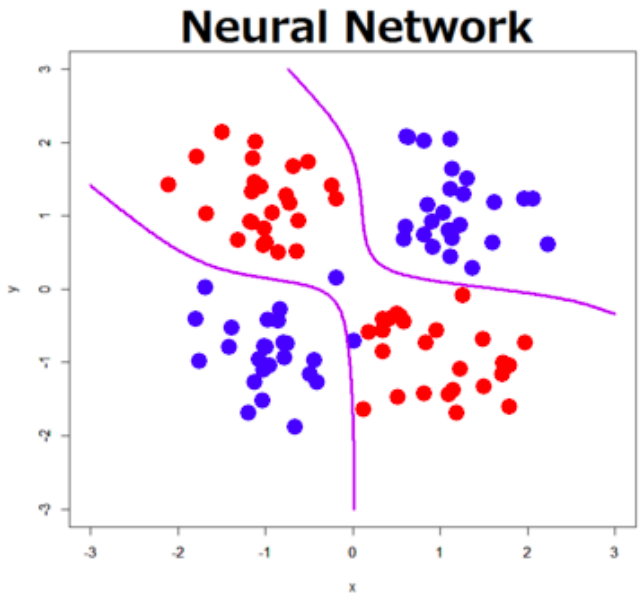

**Linearly separable pattern: 3-classes classification**

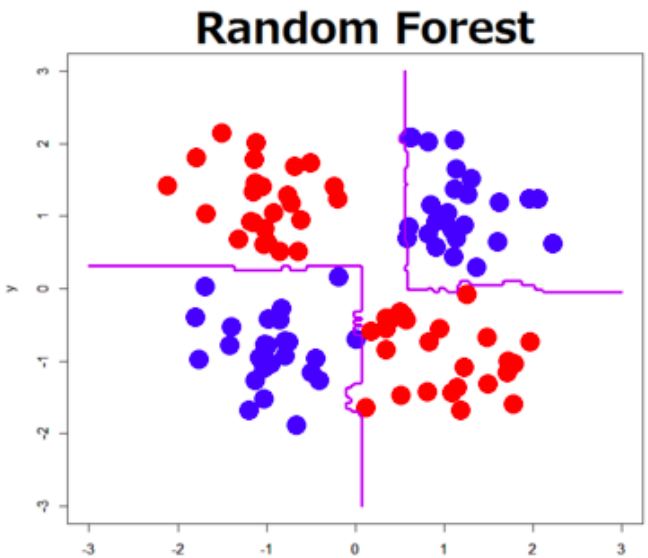


http://tjo-en.hatenablog.com



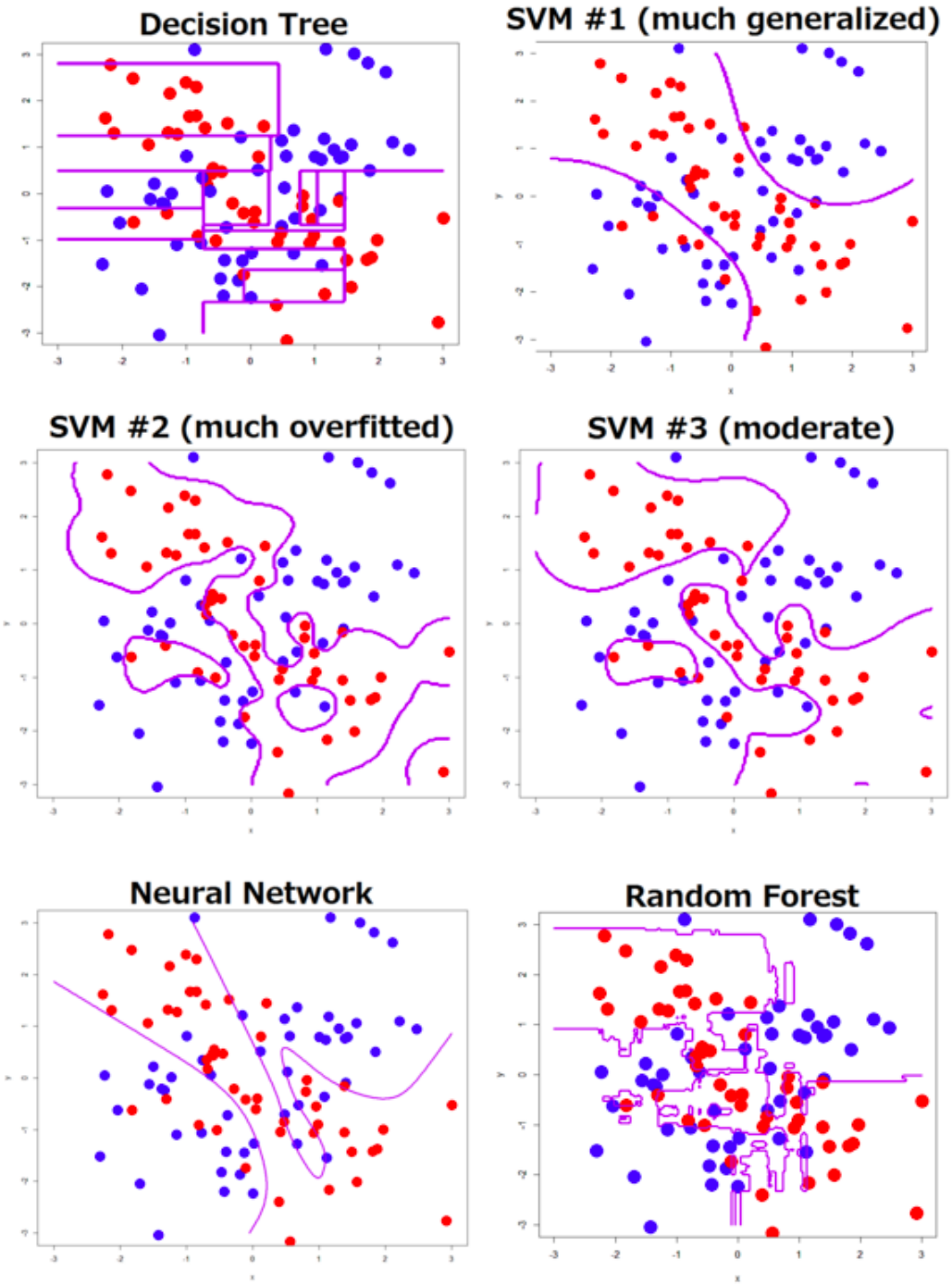




**Linearly inseparable pattern: binary classification for a simple XOR pattern**



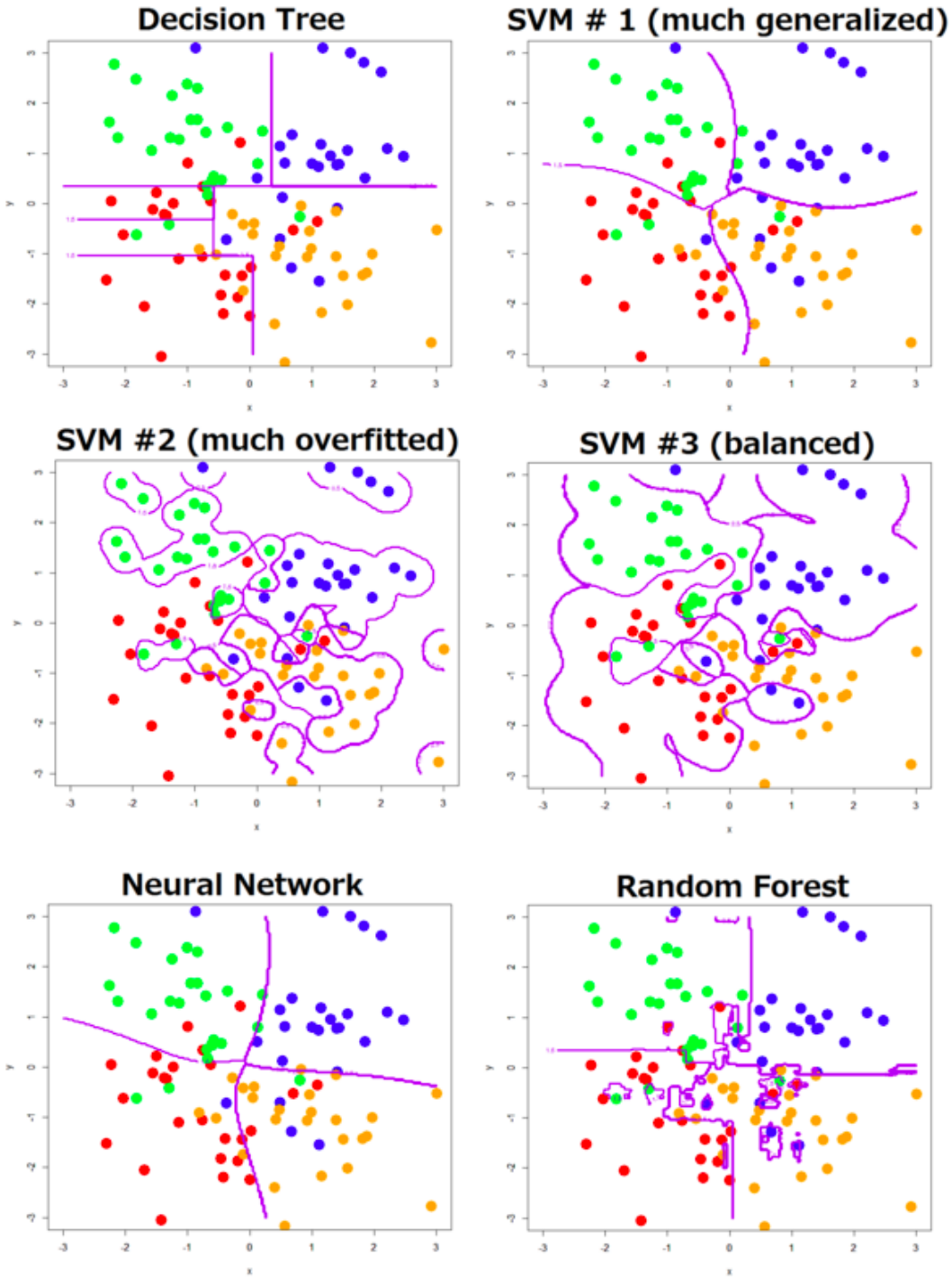
http://tjo-en.hatenablog.com/entry/2014/01/06/234155

**Linearly inseparable pattern: binary classification for a complex XOR pattern**




http://tjo-en.hatenablog.com/entry/2014/01/06/234155

# 4-classes classification for a complex pattern



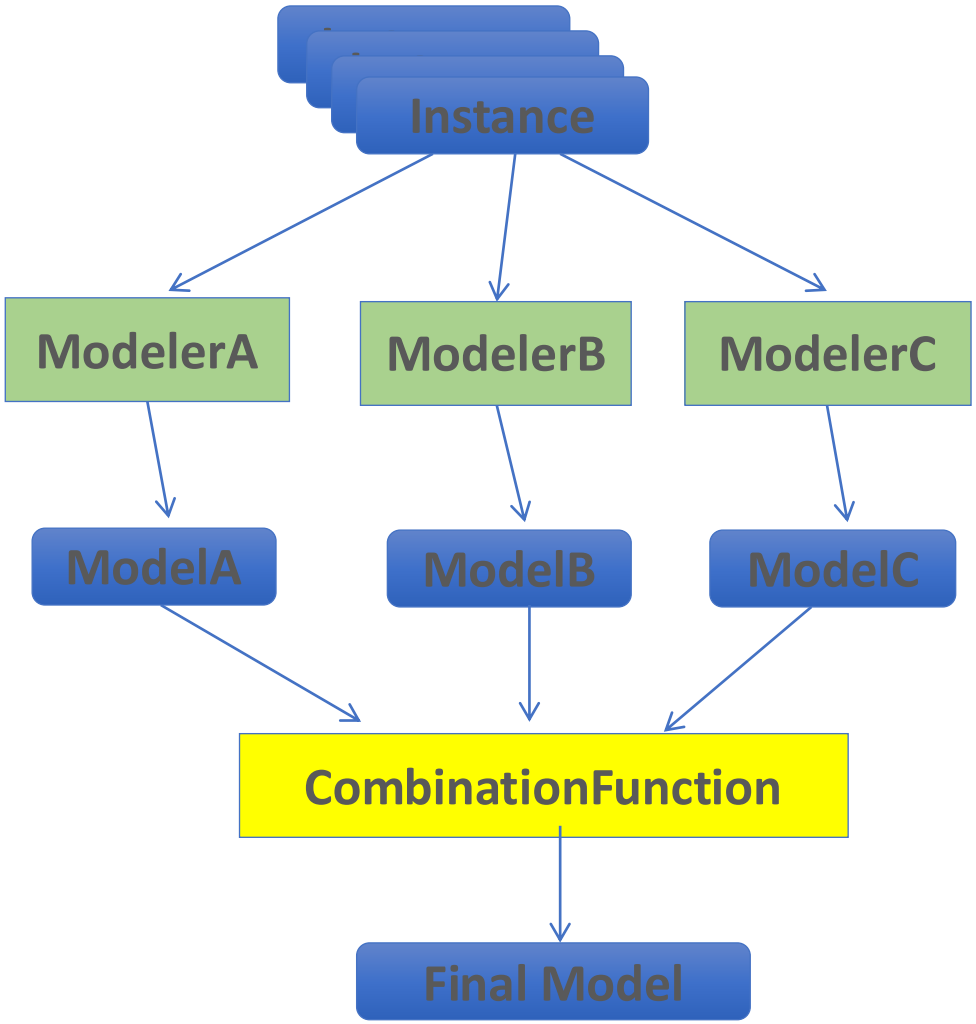

http://tjo-en.hatenablog.com/entry/201

# What Modeler to Choose?

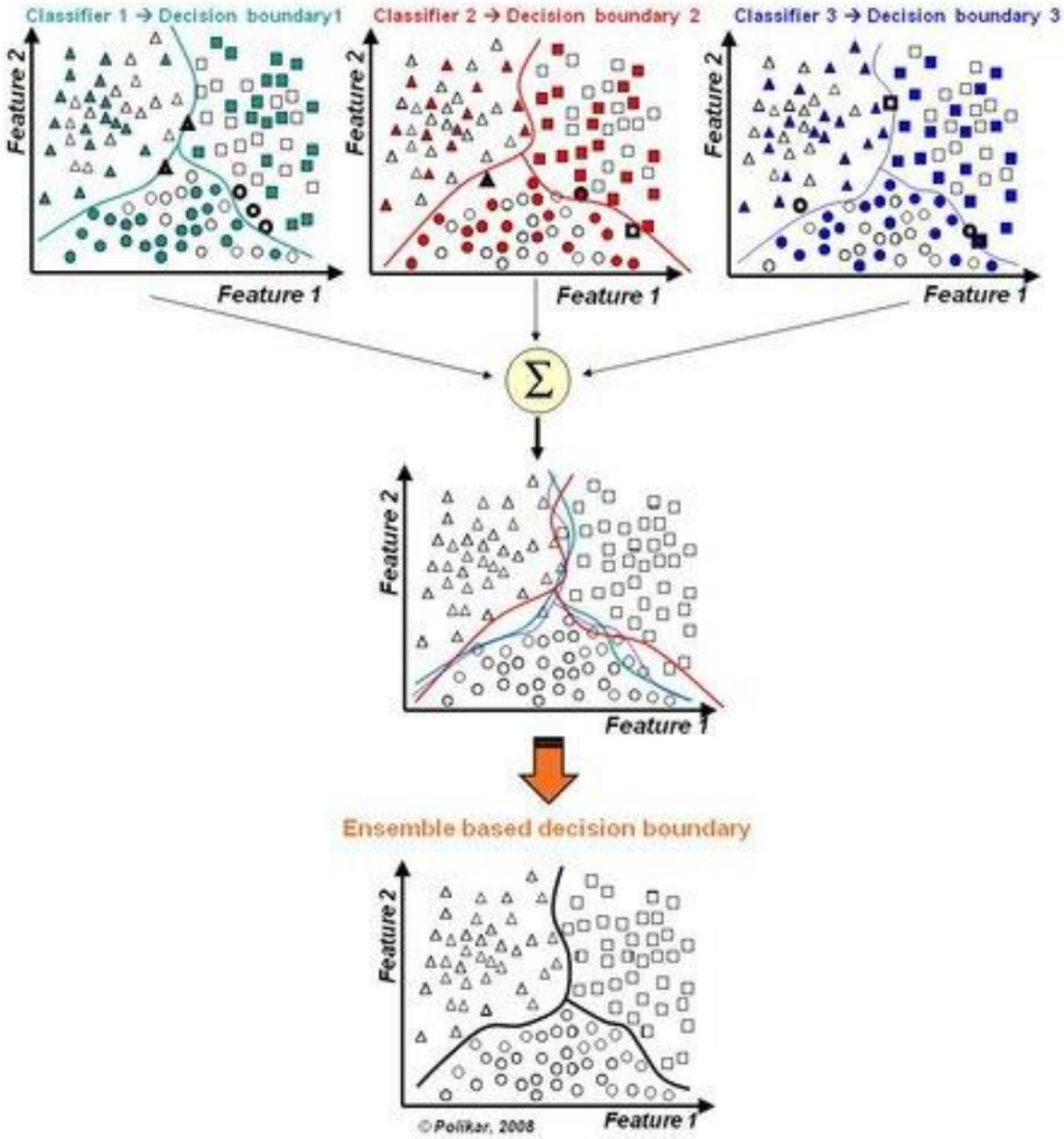
- Logistic regression
- Naïve Bayes classifiers
- Support vector machines (SVMs)
- Decision trees
- Random forests
- Kernel methods
- Genetic algorithms (GAs)
- Neural networks: perceptrons

- Data scientists **try different modelers**, with **different parameters**,
- and check the **accuracy** to figure out which one works best for the data at hand.

# Ensembles

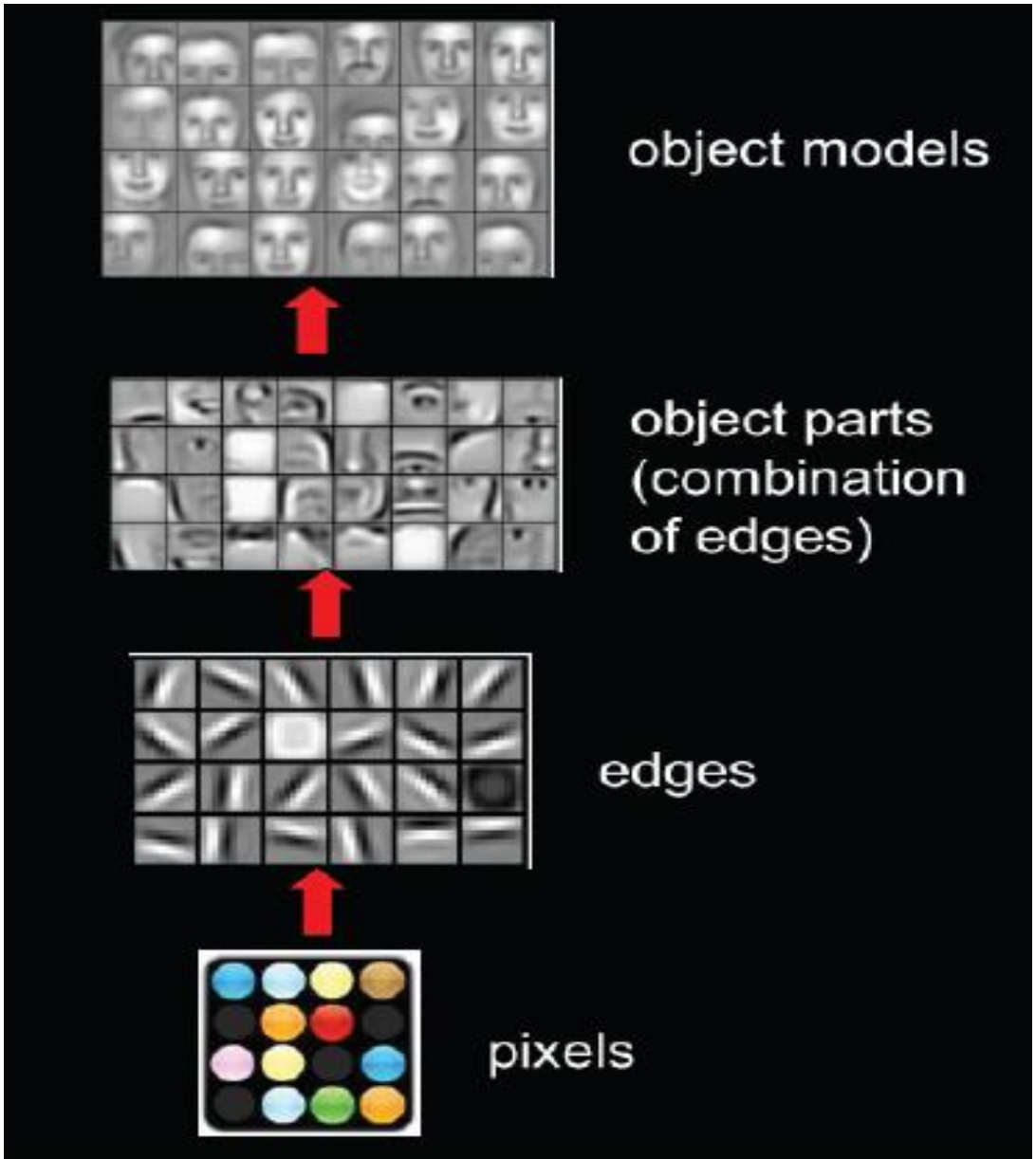



- An ensemble method uses several algorithms that do the same task, and combines their results
  - "Ensemble learning"
- A combination function joins the results
  - Majority vote: each algorithm gets a vote
  - Weighted voting: each algorithm's vote has a weight
  - Other complex combination functions.





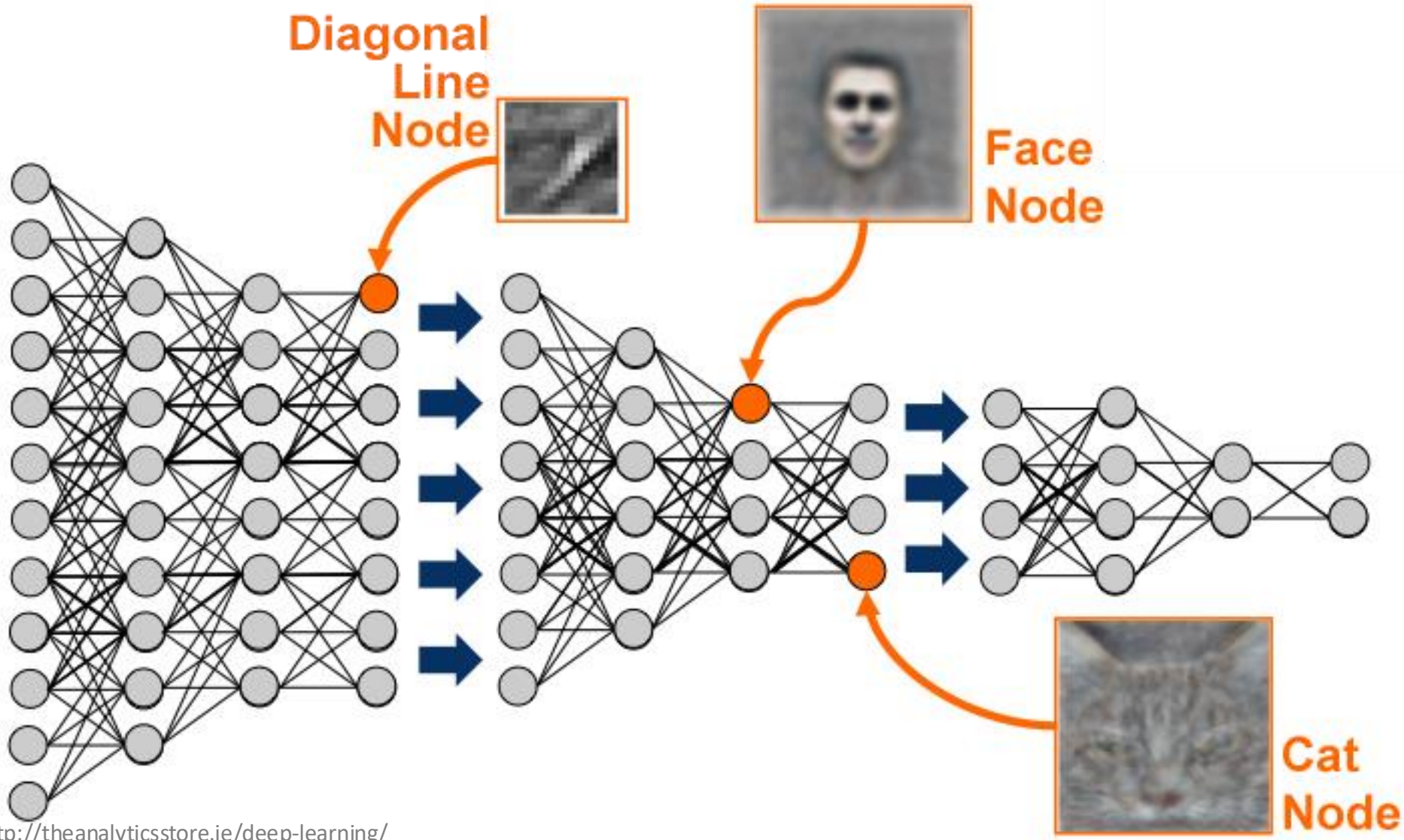

http://magizbox.com/index.php/machine-learning/ds-model-building/ensemble/




https://indico.io/blog/wp-content/uploads/2016/02/cnn_deeper.jpg

# Computer Vision

# Neural Networks and “Deep Learning”




http://theanalyticsstore.ie/deep-learning/

# Deep Learning in AlphaGo Zero

nature International weekly journal of science

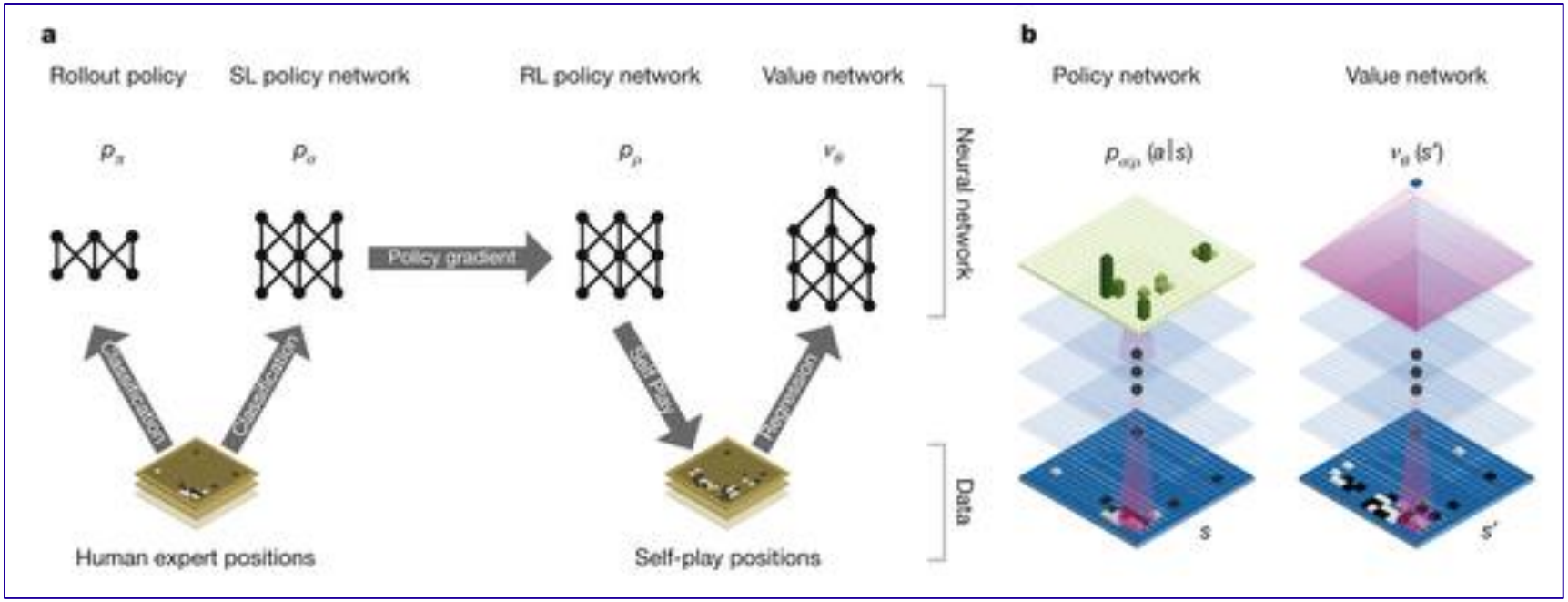
## Mastering the game of Go with deep neural networks and tree search

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel & Demis Hassabis

Affiliations | Contributions | Corresponding authors

*Nature* **529**, 484–489 (28 January 2016) | doi:10.1038/nature16961
Received 11 November 2015 | Accepted 05 January 2016 | Published online 27 January 2016

Neural network training pipeline and architecture




D Silver *et al. Nature* **529**, 484–489 (2016) doi:10.1038/nature16961

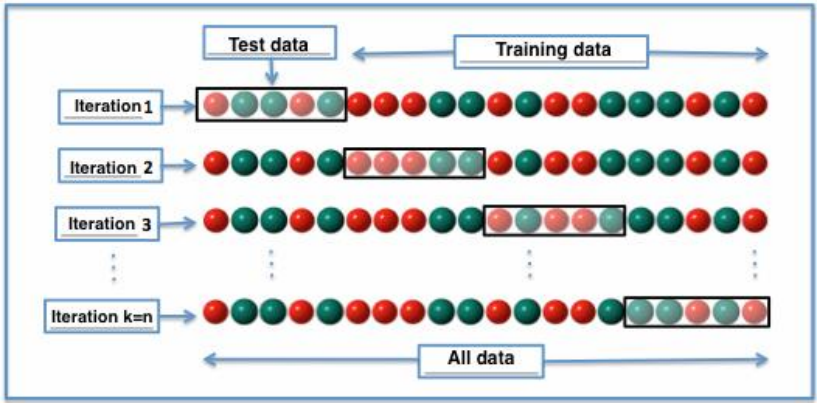
# Classification Accuracy

- Accuracy: percentage of correct classifications

$$\text{Accuracy} = \frac{\text{Total test instances classified } \textbf{correctly}}{\text{Total number of test instances}}$$

# Evaluating a Classifier: n-fold Cross Validation

- Suppose m labeled instances
  - Divide into n subsets (“*folds*”) of equal size
- Run classifier n times, with each of the subsets as the test set
  - The rest (n-1) for training
  - Each run gives an accuracy result




Translated from image by Joan.domenech91 (Own work) [CC BY-SA 3.0 (http://creativecommons.org/licenses/by-sa/3.0)], via Wikimedia Commons (https://commons.wikimedia.org/wiki/File:K-fold_cross_validation.jpg)

# Evaluating a Classifier:
# Confusion Matrix

| | Classified positive | Classified negative |
|---|---|---|
| Actual positive | **True positive** | **False negative** |
| Actual negative | **False positive** | **True negative** |

***TP: number of positive examples classified correctly***
*FN: number of positive examples classified incorrectly*
*FP: number of negative examples classified incorrectly*
***TN: number of negative examples classified correctly***

# Evaluating a Classifier: Precision and Recall

***TP: number of positive examples classified correctly***
*FN: number of positive examples classified incorrectly*
*FP: number of negative examples classified incorrectly*
***TN: number of negative examples classified correctly***

$$\textbf{Precision} = \frac{TP}{TP + FP} \qquad \textbf{Recall} = \frac{TP}{TP + FN}$$

Note that the focus is on the **positive class**

# Evaluating a Classifier: Other Metrics

- Other accuracy metrics
  - F1-score
  - Receive Operating Characteristics (ROC) curve
  - Area Under the Curve (AUC)
- Other concerns
  - Explainability of classifier results
    - ability to *explain* why certain predictions are made
  - Cost of examples
    - Cost of feature values
    - Labeling

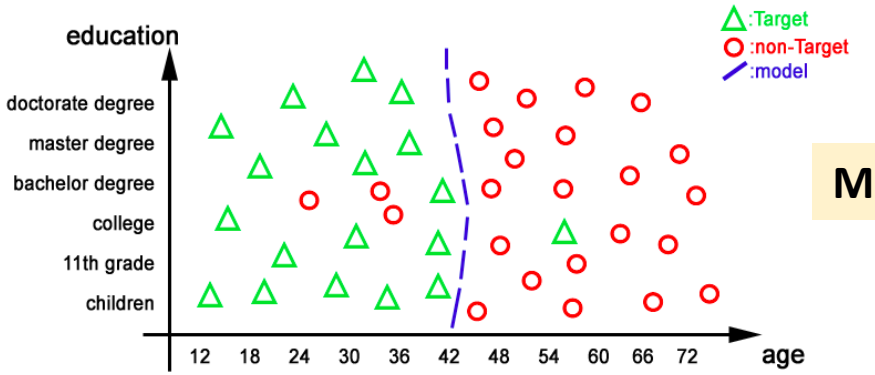
# Evaluating a Classifier: What Affects the Performance

- **Few instances** for a complex classification task
- **Missing feature** values for instances
- **Errors in attribute values** for instances
- **Errors in the labels** of training instances
- **Uneven availability** of instances in classes.

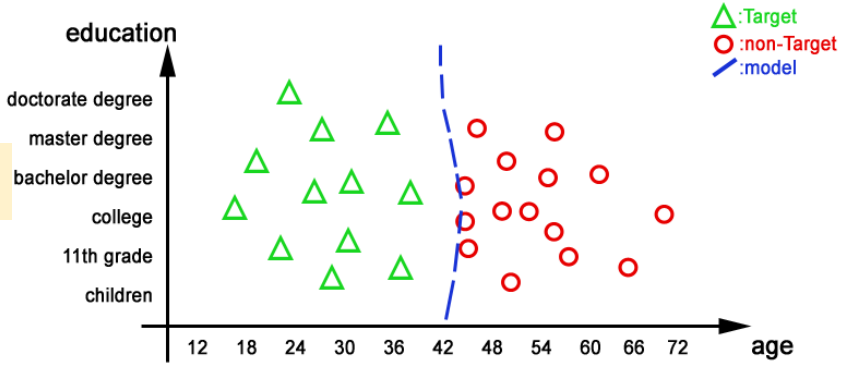
# Overfitting

- A model overfits the **training data** when it is **very accurate** with that data, and may not do so well with **new test data**

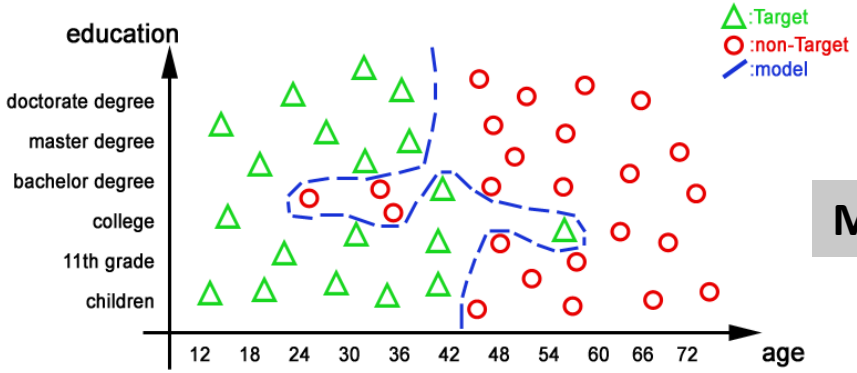
**Training Data**



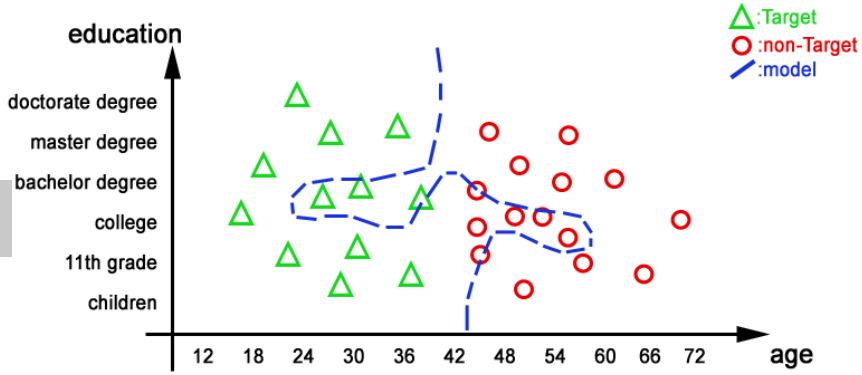

**Test Data**




**Model 1**




**Model 2**

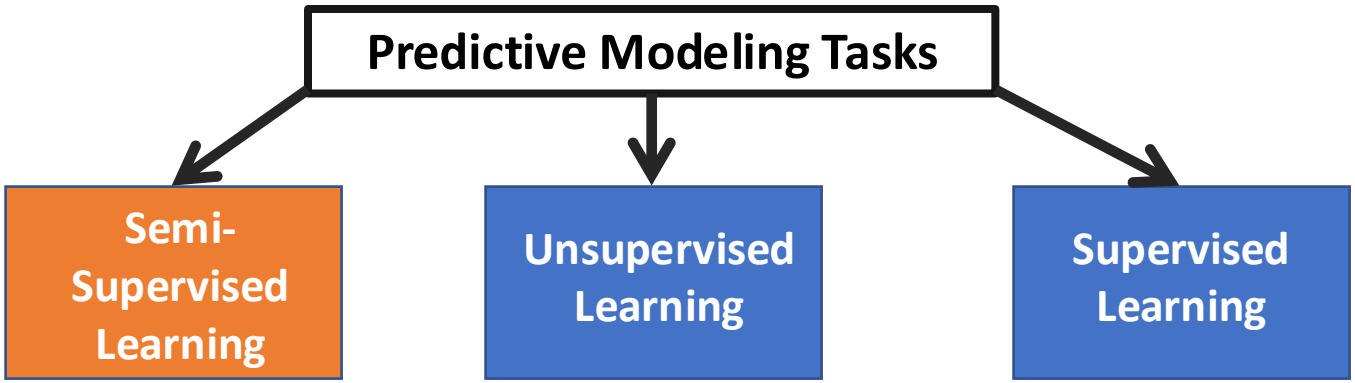


# Summary of Concepts in Classification

- Instances, features, values
- Classes, disjoint classes
- Labels, binary tasks
- Learning
  - Decision trees
  - Modeler
  - Ensembles, combination function
    - Majority vote, weighted vote
- Induction

- Training and test sets
- Evaluation
  - Accuracy, confusion matrix, precision & recall
  - N-fold cross validation
  - Overfitting
- About the data
  - High dimensionality
  - Sparse data
  - Continuous/discrete values
  - Latent variables

# Clustering vs. Classification

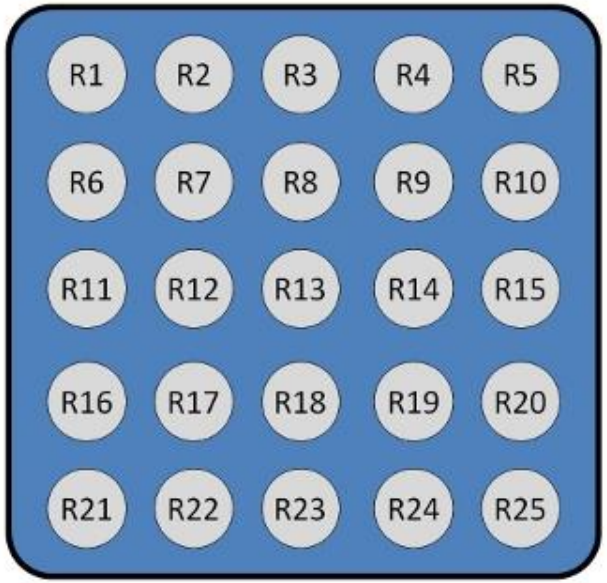



- Classification is **supervised**
  - class labels are provided;
  - learn a **classifier to predict class labels** of novel/unseen data
- Clustering is **unsupervised** or **semi-supervised**;
  - No class label is give
  - Understand the structure underlying your data

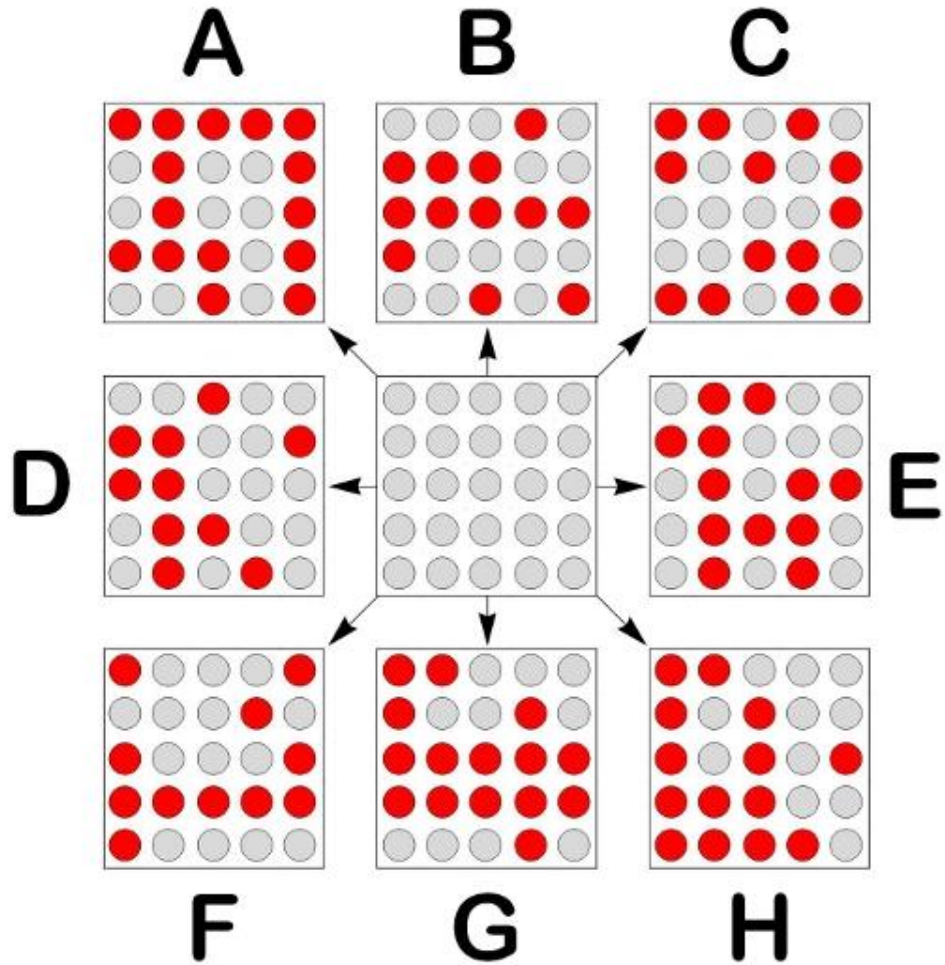
# Different Data Analysis Tasks

- **Classification**
  - Assign a category (ie, a class) for a new instance
- **Clustering**
  - Form clusters (ie, groups) with a set of instances
- **Pattern discovery**
  - Identify regularities (ie, patterns) in temporal or spatial data
- **Simulation**
  - Define mathematical formulas that can generate data similar to observations collected

# Spatial Patterns



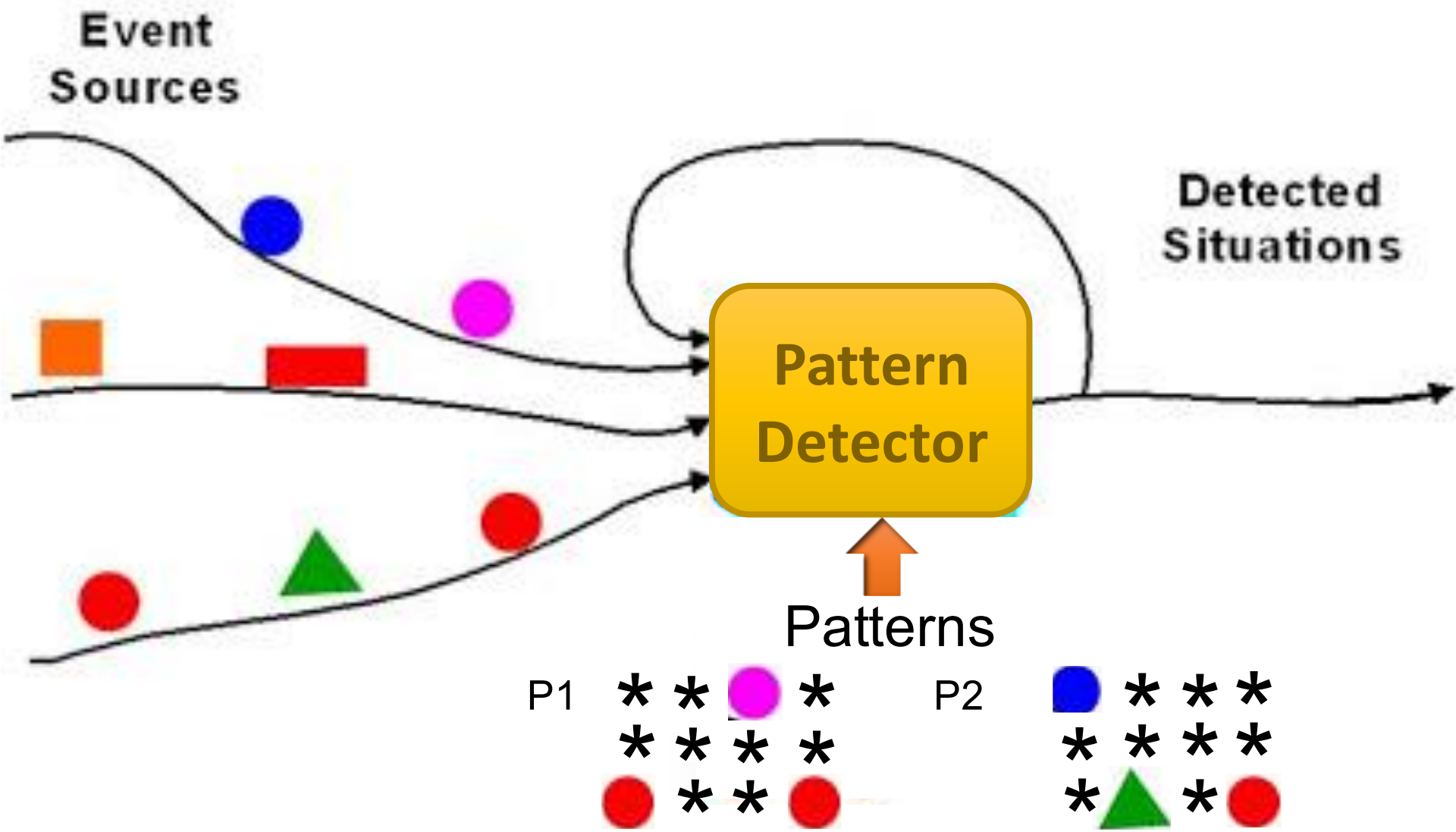

Patterns




**Patterns** may be recognized because of their arrangement.

http://bama.ua.edu/~mbonizzoni/research.html

# Temporal Patterns






http://epthinking.blogspot.com/2009/01/on-event-pattern-detection-vs-event.html

# Detecting Patterns in a Text String

- abababababab
- abcabcabcabc
- abcccccccabcccabcccccccccccabcabccc

# A Pattern Language

- abababababab
  - (ab)*
- abcabcabcabc
  - (abc)*
- abcccccccabcccabcccccccccccabcabccc
  - ((ab)(c)*)*

# Detecting Patterns in Streaming Data

- (ab)*x*
  - Abababthsrthwababyertueyrtyertheabsgd
- abcabcabcabc
  - abcabcrgkskhgsnrhnabcabcabcabcrjgjsrn

# Concept Drift

- Over time, the **data source changes** and the concepts that were **learned in the past** have now changed
- The model built on old data **inconsistent** with the new data,
  - The predictions become less accurate as time passes
  - Regular updating of the model is necessary
  - Eg. weather prediction

# Pattern Detection vs Pattern Learning

## Pattern Detection

- Inputs:
  - Data
  - A **set of patterns**
- Output:
  - Matches of the patterns to the data

## Pattern Learning

- Inputs:
  - **Data annotated** with a set of patterns
- Output:
  - A set of patterns that appear in the data with some frequency

# Pattern Detection vs Pattern Learning

## Pattern Learning

- Inputs:
  - Data annotated with a set of patterns
- Output:
  - A set of patterns that appear in the data with some frequency

## Pattern Discovery

- Inputs:
  - Data
- Output:
  - A set of patterns that appear in the data with some frequency

# Pattern Detection vs. Pattern Discovery

- Pattern detection
  - Predefined patterns has occurred.
- Pattern discovery
  - Pattern is unknown in advance
  - Discovery function determines the pattern.

# Pattern Detection vs. Pattern Discovery

**Scenarios:**

**Stock Trading**

- **Automatic identification of buy/sell opportunities.**

**Compliance Checks**

- **Sarbanes-Oxley detection.**

**Fraud Detection**

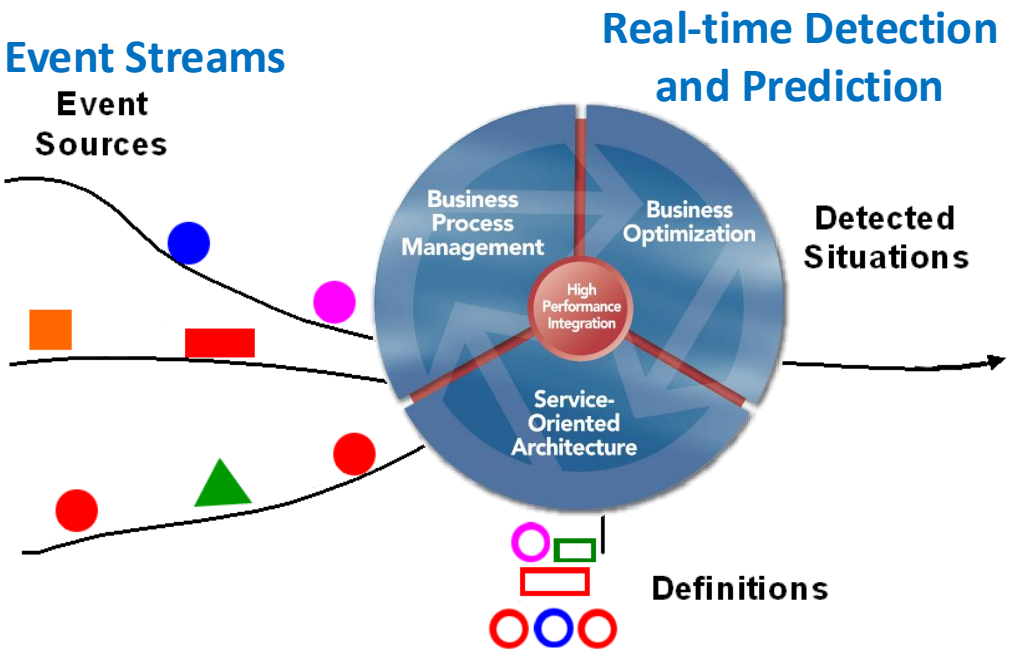
- **Odd credit card purchases performed within a period.**

**CRM**

- **Alert if three orders from the same platinum customer were rejected.**

**Insurance Underwriting**

- **Identification of risk.**




*"Events in several forms, from simple events to complex events, will become very widely used in business applications during 2004 through 2008"*

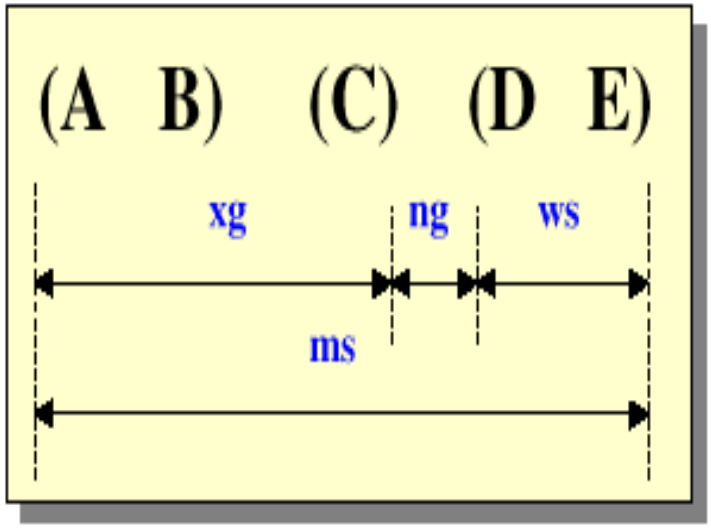
--- Gartner July 2003

Graphic Sources: TIBCO Software Inc & IBM

# Sequential Pattern Discovery

- Given: **set of objects**, each associated with its own **timeline of events**, **find rules** that **predict strong sequential dependencies** among different events, of the form

  (A B) (C) (D E) --> (F)




- xg :max allowed time between consecutive event-sets
- ng: min required time between consecutive event sets
- ws: window-size, max time difference between earliest and latest events in an event-set (events within an event-set may occur in any order)
- ms: max allowed time between earliest and latest events of the sequence.

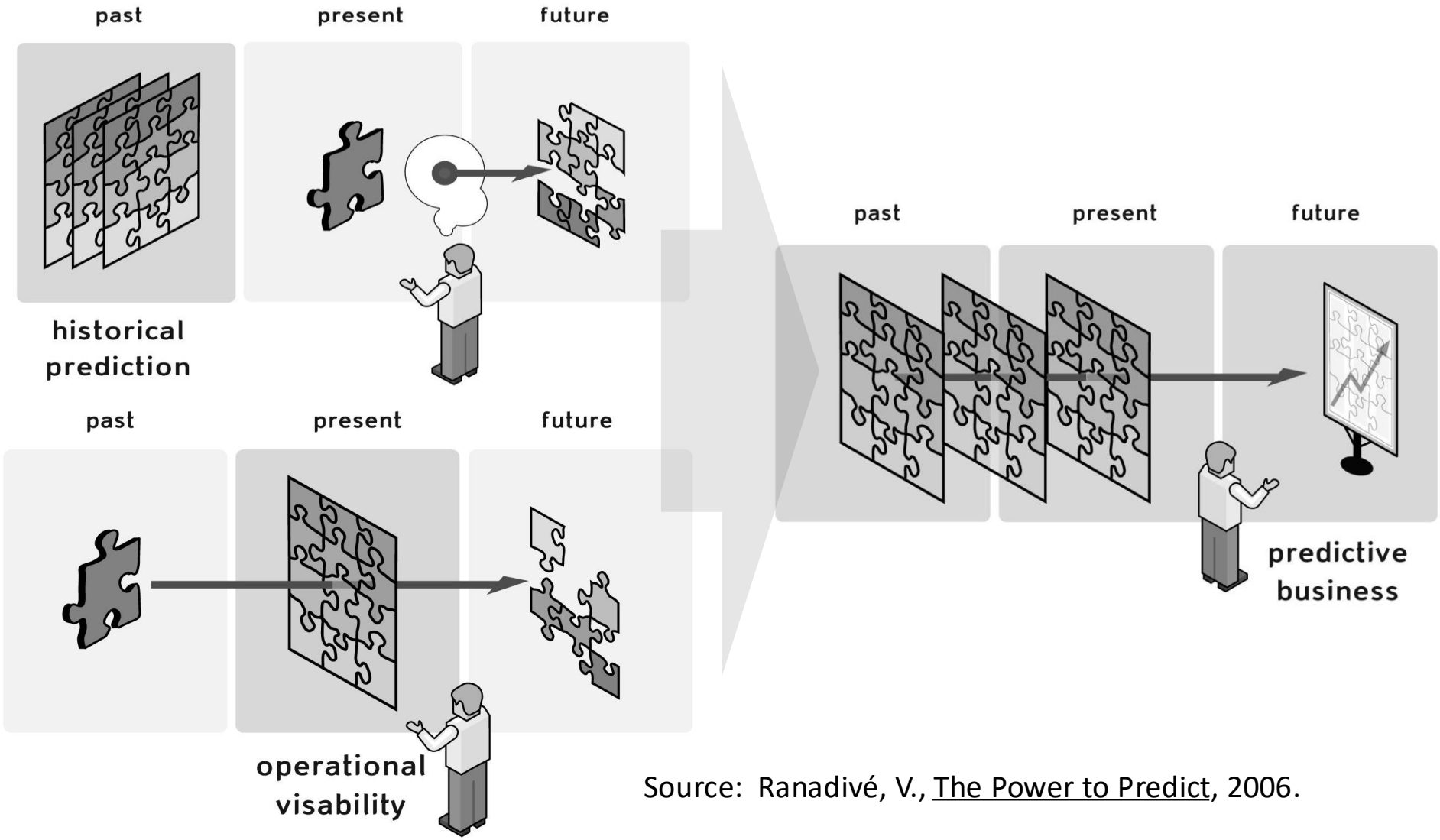
# Sequential Pattern Discovery: Examples

- Sequences Customers purchase goods/services
- Understanding long term behavior
- In point-of--sale transaction sequences
  - Computer bookstore:

    (Intro to Visual C++) (C++ Primer) --> (Perl for Dummies, TCL/TK)
  - Athletic Apparel Store:

    (Shoes) (Racket, Racketball) --> (Sports Jacket).

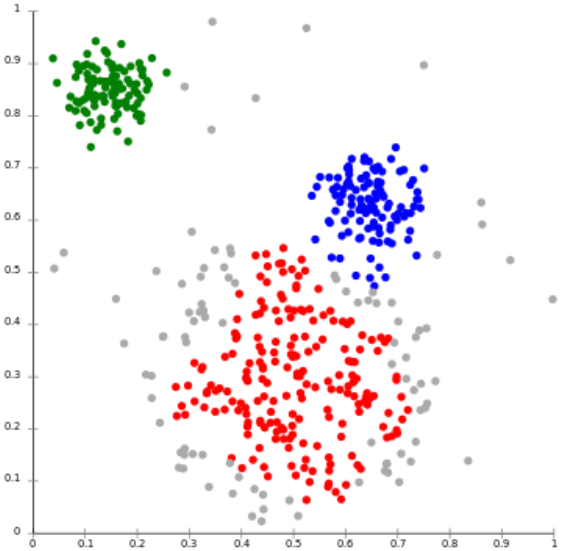
# Application: Predictive Business




Source: Ranadivé, V., The Power to Predict, 2006.

# Clustering

- Given:
  - A set of instances (datapoints), with feature values
    - Feature vectors
  - A target number of clusters (k)
- Find:
  - The “best” assignment of instances (datapoints) to clusters
    - “Best”: satisfies some optimization criteria
    - “clusters” represent similar instances




https://commons.wikimedia.org/wiki/File:DBSCAN-Gaussian-data.svg

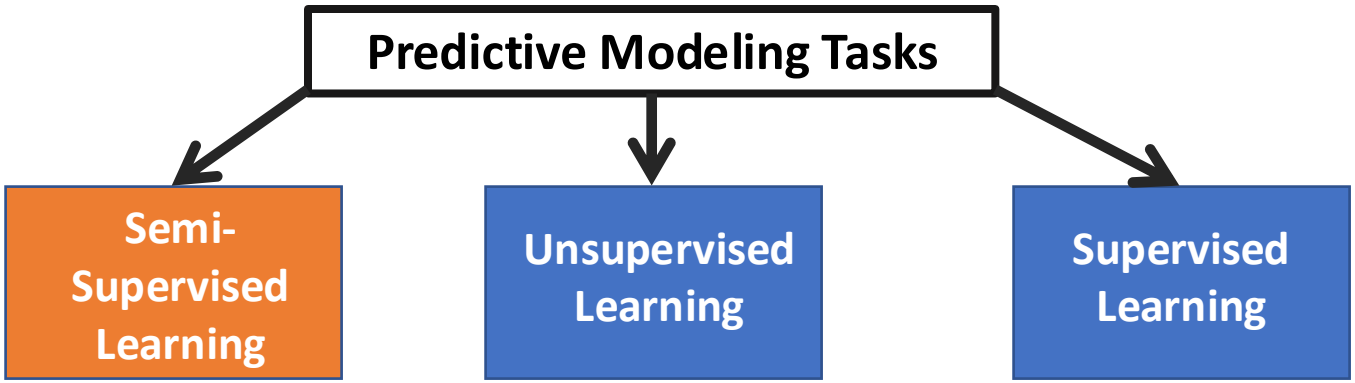
# High Dimensional Data

- Given a cloud of data points we want to understand its **structure**
- Group points into **“clusters”** according to some **distance measure**.

# The Problem of Clustering

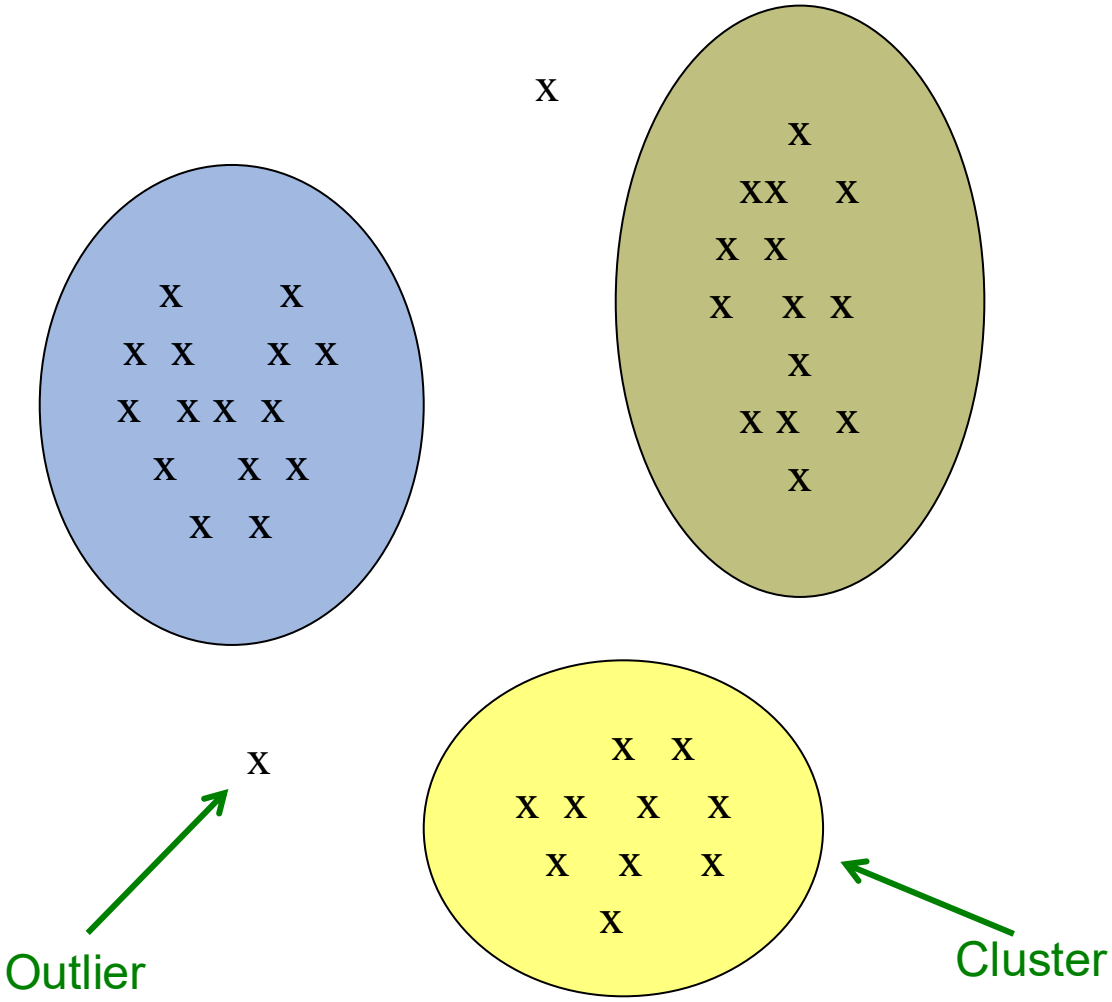
- Given a **set of points** that belong to some **space**, with a notion of **distance** between points
- **Group the points** into some number of ***clusters***, so that:
  - **Members** of a cluster are **close/similar to** each other
  - **Members** of different clusters are **dissimilar**
- **Usually:**
  - Points are in a high-dimensional space
  - Similarity is defined using a **distance measure**
    - Euclidean, Cosine, Jaccard, edit distance, …

# Clustering vs. Classification

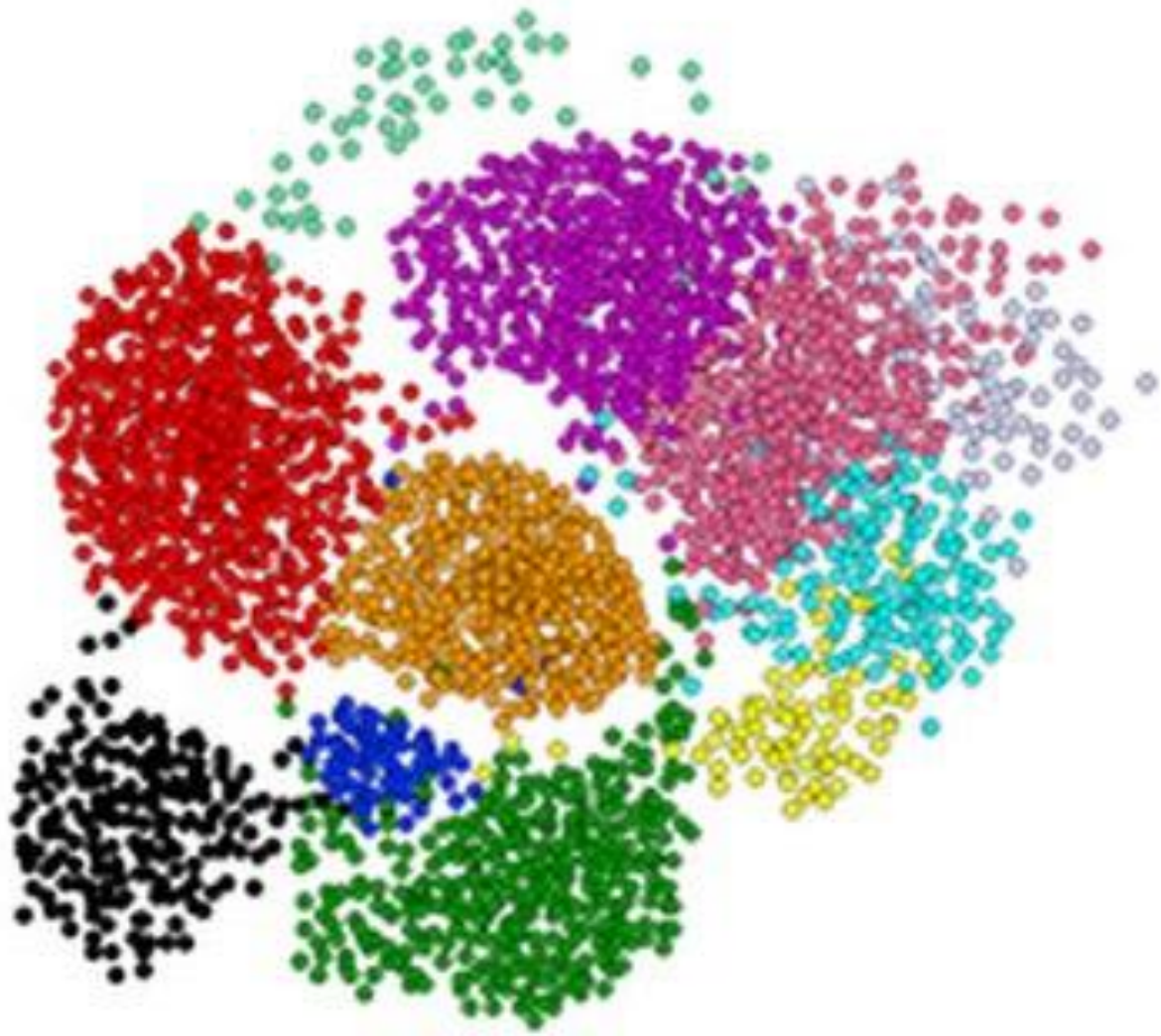



- Classification is **supervised**
  - class labels are provided;
  - learn a **classifier to predict class labels** of novel/unseen data
- Clustering is **unsupervised** or **semi-supervised**;
  - No class label is give
  - Understand the structure underlying your data

# Example: Clusters & Outliers



# Clustering is a hard problem!



# Clustering Problem: Songs

- **Intuitively: Music divides into categories, and customers prefer a few categories**
  - But what are categories really?
- Represent a song by a **set of customers** who bought it
- **Similar songs** have similar sets of customers, and vice-versa.

# Clustering Problem: Songs

**Space of all songs:**

- Think of a space with one dimension for each customer
  - Values in a dimension may be 0 or 1 only
  - A song is a point in this space $(x_1, x_2, \ldots, x_k)$, where $x_i = 1$ iff the $i^{\text{th}}$ customer bought the song
- For Amazon, the dimension is tens of millions
- **Task:** Find clusters of similar songs.

# Clustering Problem: Documents

**Finding topics:**

- Represent a document by a vector ($x_1$, $x_2$,…, $x_k$), where $x_i$ = 1 iff the $i$ th word (in some order) appears in the document
  - It actually doesn't matter if $k$ is infinite; i.e., we don't limit the set of words
- **Documents with similar sets of words may be about the same topic.**

# Euclidean, Cosine, and Jaccard Distance

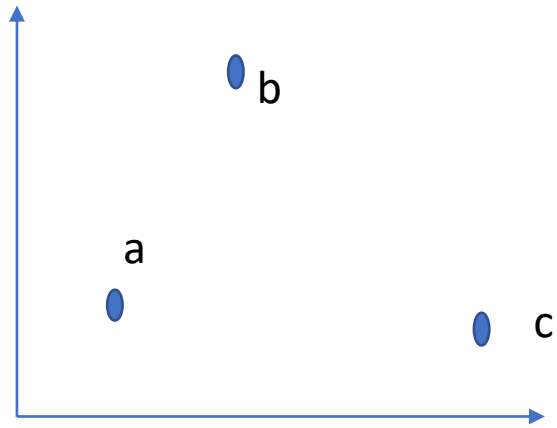
**We have a choice when we think of documents as sets of words or shingles:**

- **Sets as points:** Measure similarity by **Euclidean distance**
- **Sets as vectors:** Measure similarity by the **cosine distance**
- **Sets as sets:** Measure similarity by the **Jaccard distance.**

# Sets as points: Euclidean Distance

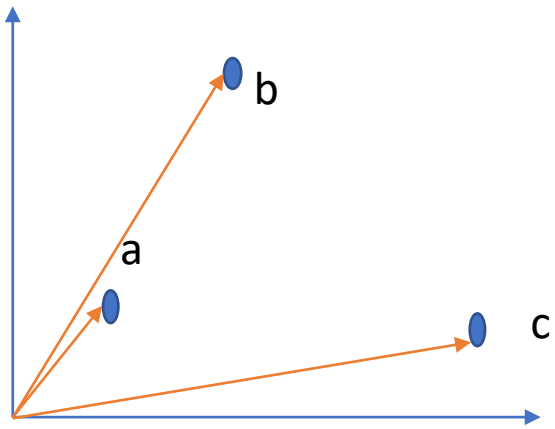
- Measures distance of two points in Euclidean space

$$d([x_1, x_2, \ldots, x_n],\ [y_1, y_2, \ldots, y_n]) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

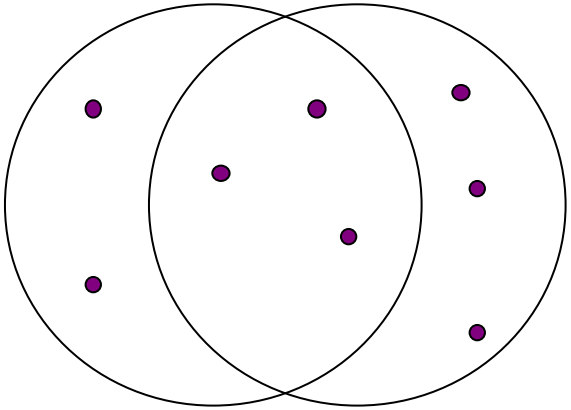


# Sets as vectors: Cosine Distance

- Similarity = Cosine of angle btw vectors: A & B
- distance = 1- Cosine(A, B)



# For Sets, use Jaccard Distance



3 in intersection
8 in union

**Jaccard similarity** = Intersection/Union

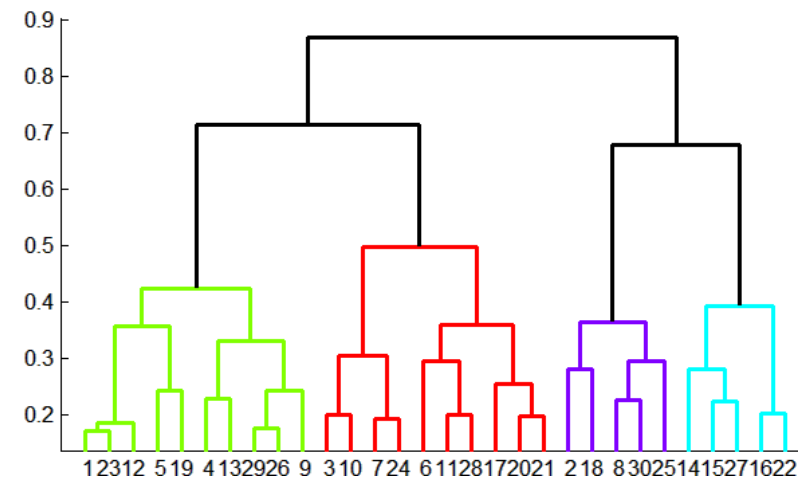
= 3/8 in this example

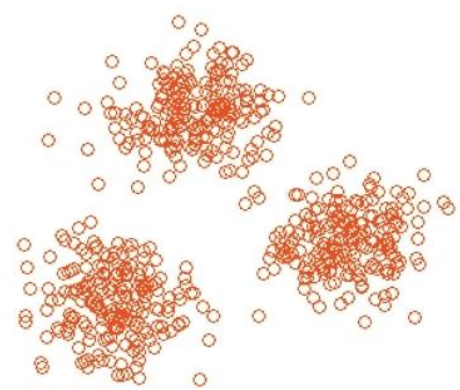
**Jaccard distance = 1 – Jaccard Similarity**

or 5/8 in this example

# Overview: Methods of Clustering

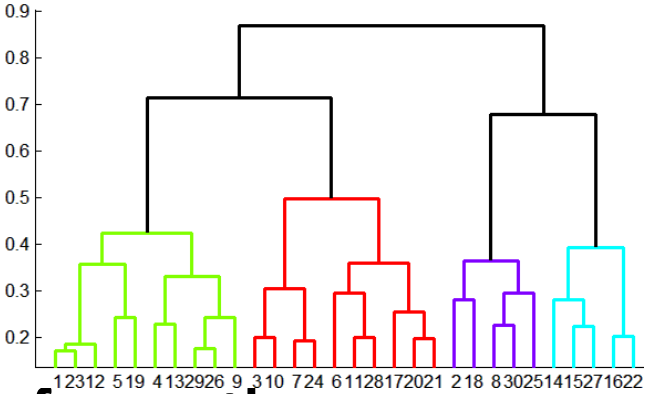
- **Hierarchical:**
  - **Agglomerative** (bottom up):
    - Initially, each point is a cluster
    - Repeatedly combine the two "nearest" clusters into one
  - **Divisive** (top down):
    - Start with one cluster and recursively split it
- **Point assignment:**
  - Maintain a set of clusters
  - Points belong to "nearest" cluster.



# Hierarchical Clustering (Agglomerative)

- **Key operation:**
  **Repeatedly combine two nearest clusters**




- **Three important questions:**
  - **1)** How do you represent a **cluster of more than one** point?
  - **2)** How do you determine the **"nearness"** of clusters?
  - **3)** When to **stop combining** clusters?

# Hierarchical Clustering

- **Key operation: Repeatedly combine two nearest clusters**
- **(1) How to represent a cluster of many points?**
  - **Key problem:** As you merge clusters, how do you represent the **"location"** of each cluster, to tell which pair of clusters is closest?
- **Euclidean case:** each cluster has a ***centroid*** **= average of its (data) points**
- **(2) How to determine "nearness" of clusters?**
  - **Measure cluster distances by distances of centroids.**

# Hierarchical Clustering

- **Initially, a point is in a cluster by itself**

How to pick and combine efficiently?

When to stop?

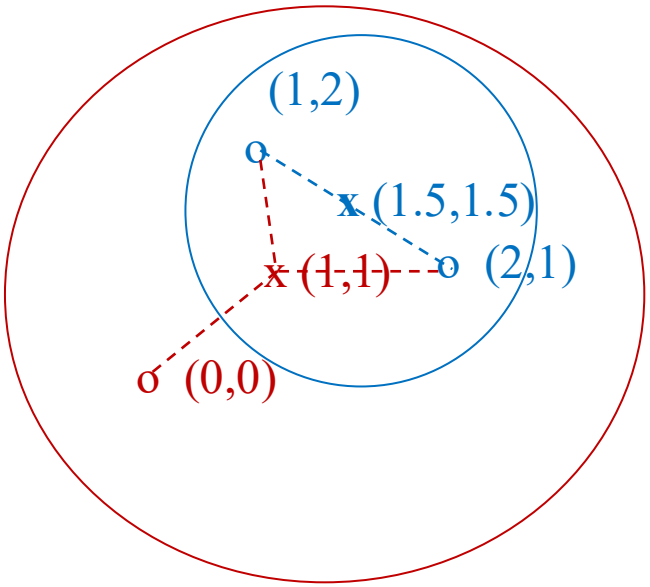
```
WHILE it is not time to stop DO
    pick the best two clusters to merge;
    combine those two clusters into one cluster;
END;
```

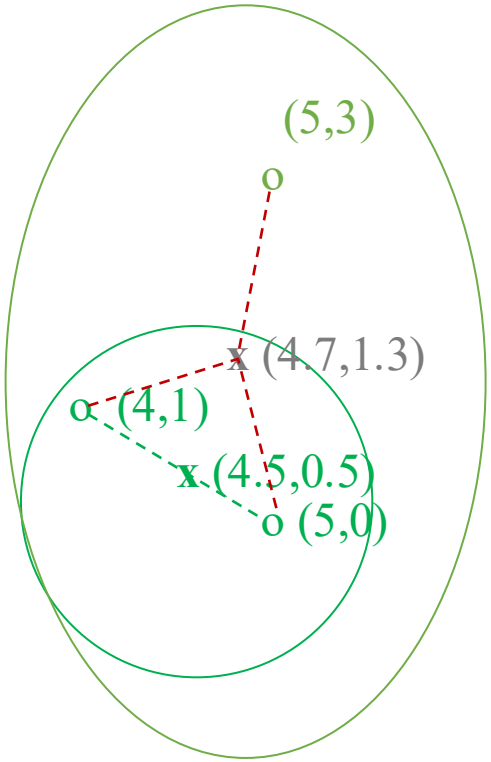
How to measure cluster distance?

# Hierarchical Agglomerative (Bottom-Up) Clustering Algorithm

- Represent cluster by its **centroid** or average of points in the cluster
- **Merging rule:**
  - the distance between two clusters is distance between their centroids
  - dist(C1, C2) = distance of their centroids
    - Coordinate of centroid = avg of that of all points in the cluster
  - C1: {(1, 2), (2, 2)}
    - Centroid = (1.5, 2)
- **merge two clusters at shortest distance.**

# Example: Hierarchical clustering

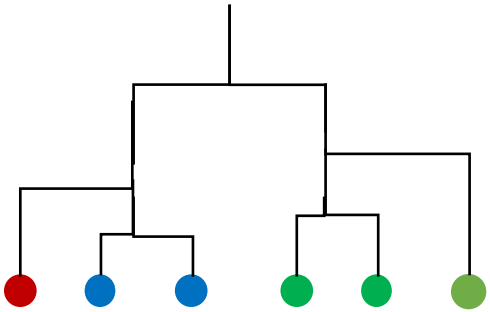



**Data:**

o … data point

x … centroid



**Dendrogram**

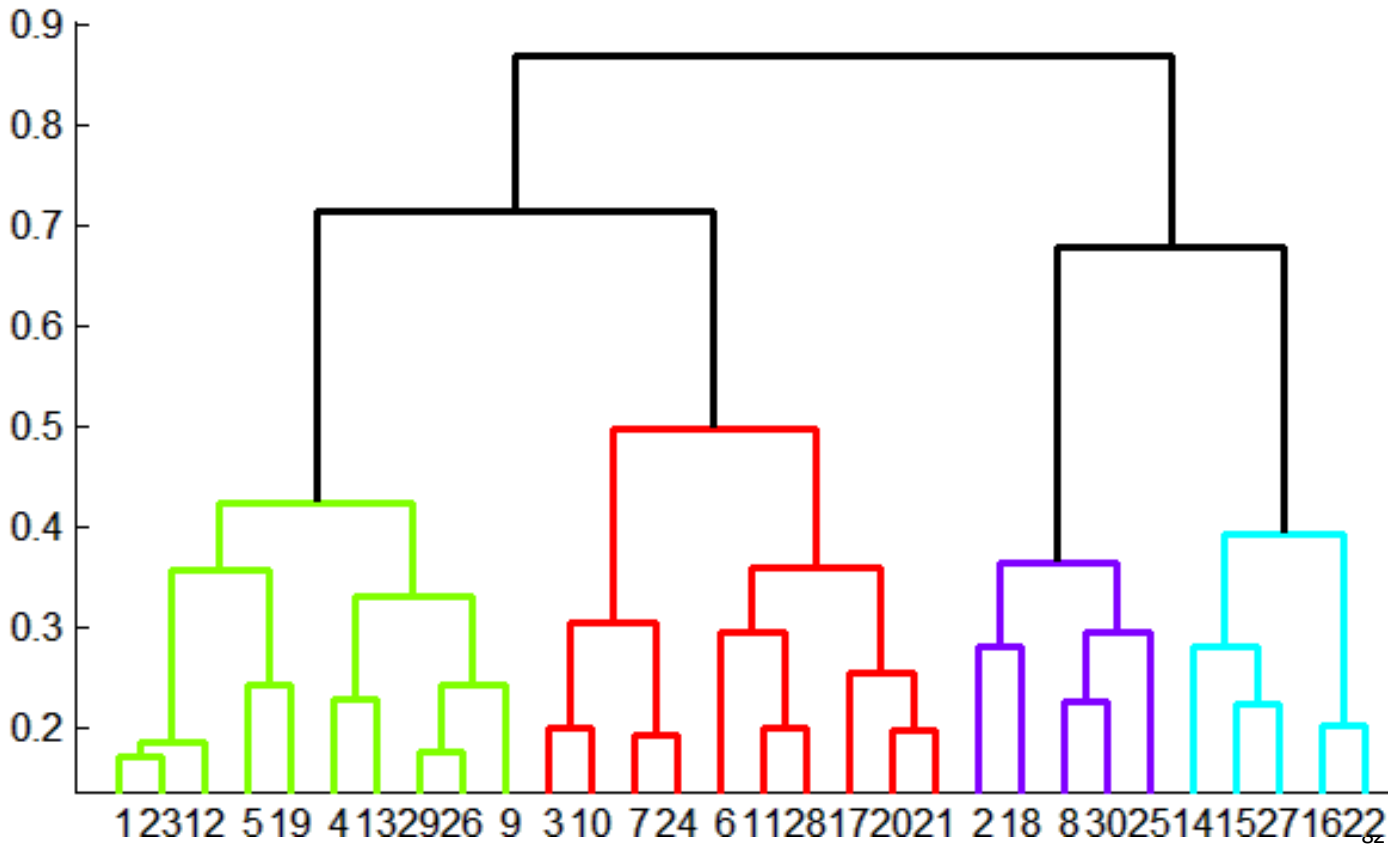
# When to Stop Clustering Process

- Several approaches:

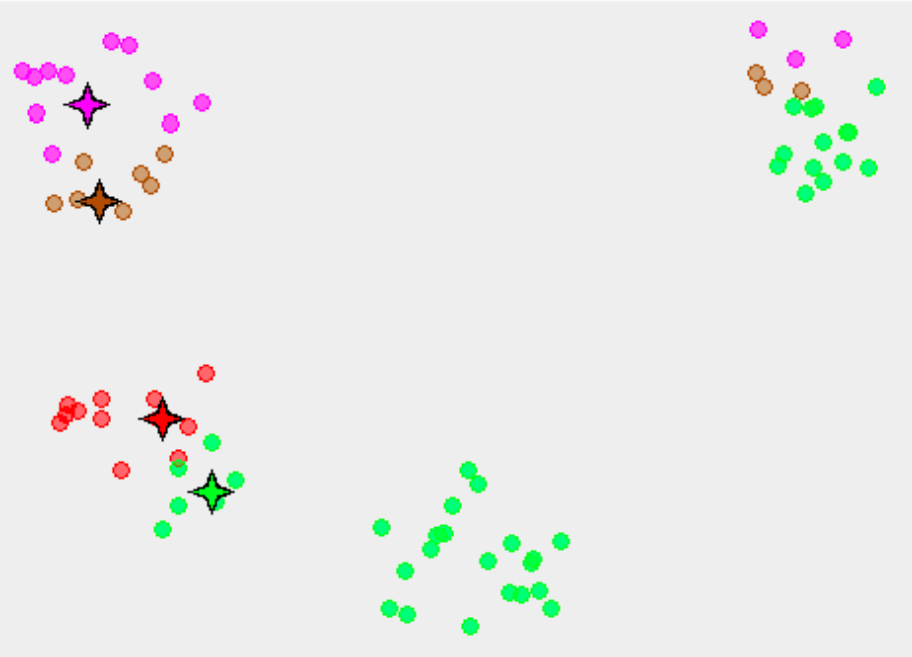
1. **May know how many clusters** there are in the data
   - Have been told or some **intuitive number of clusters**
2. Stop combining **when best combination of existing clusters produces a cluster that is inadequate**
   - E.g., Average distance between centroid and its points should be below some limit.

# Dendrogram

- **Can obtain k clusters from result for desired k**
  - **k can be any value between 1 and n**

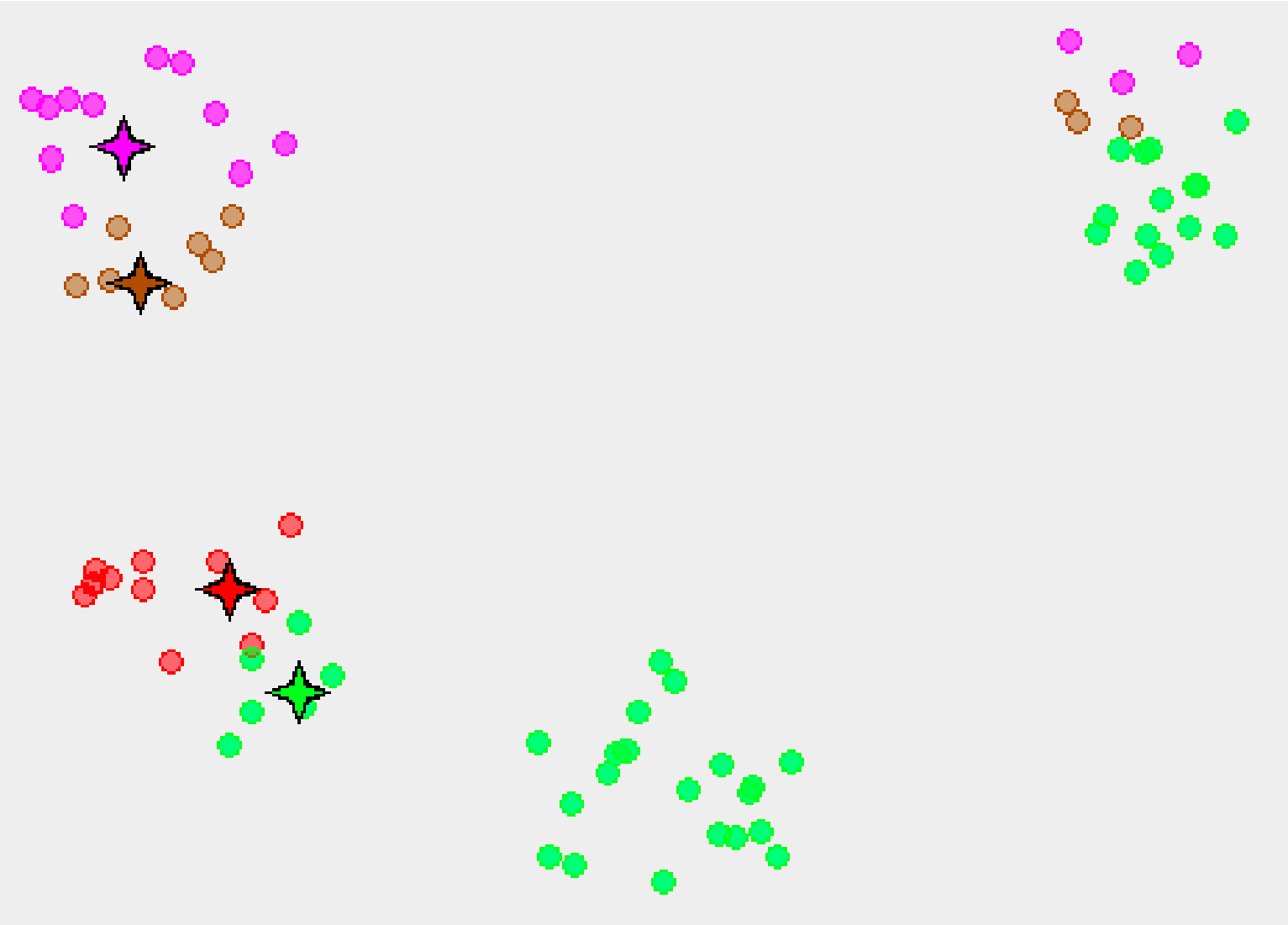


# K-Means Clustering Algorithm



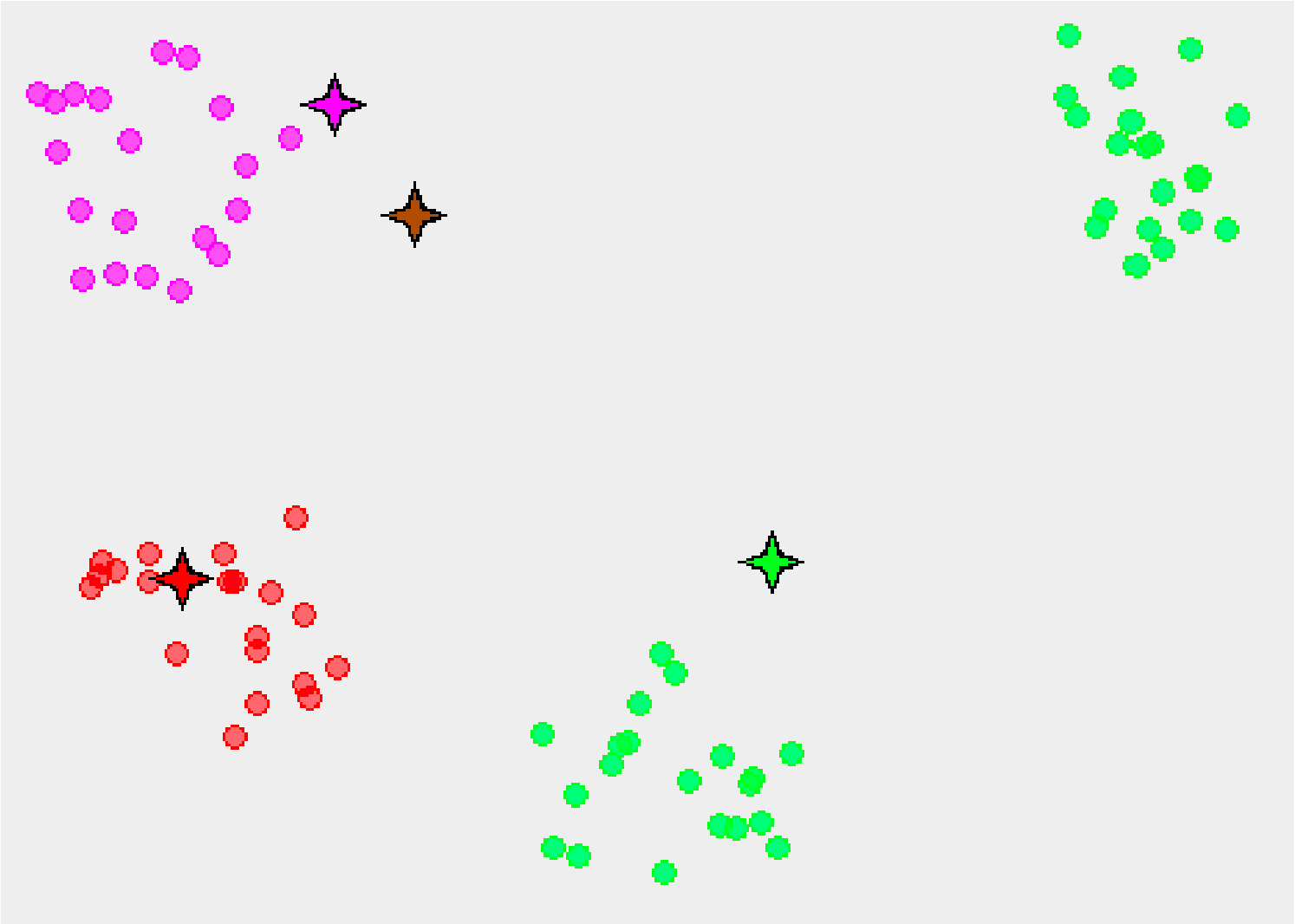
- User specifies a target number of clusters (k)
- Place randomly k cluster centers
- For each datapoint, attach it to the nearest cluster center
- For each center, find the centroid of all the datapoints attached to it
- Turn the centroids into cluster centers
- Repeat until the sum of all the datapoint distances to the cluster centers is minimized

# K-Means Clustering (1)



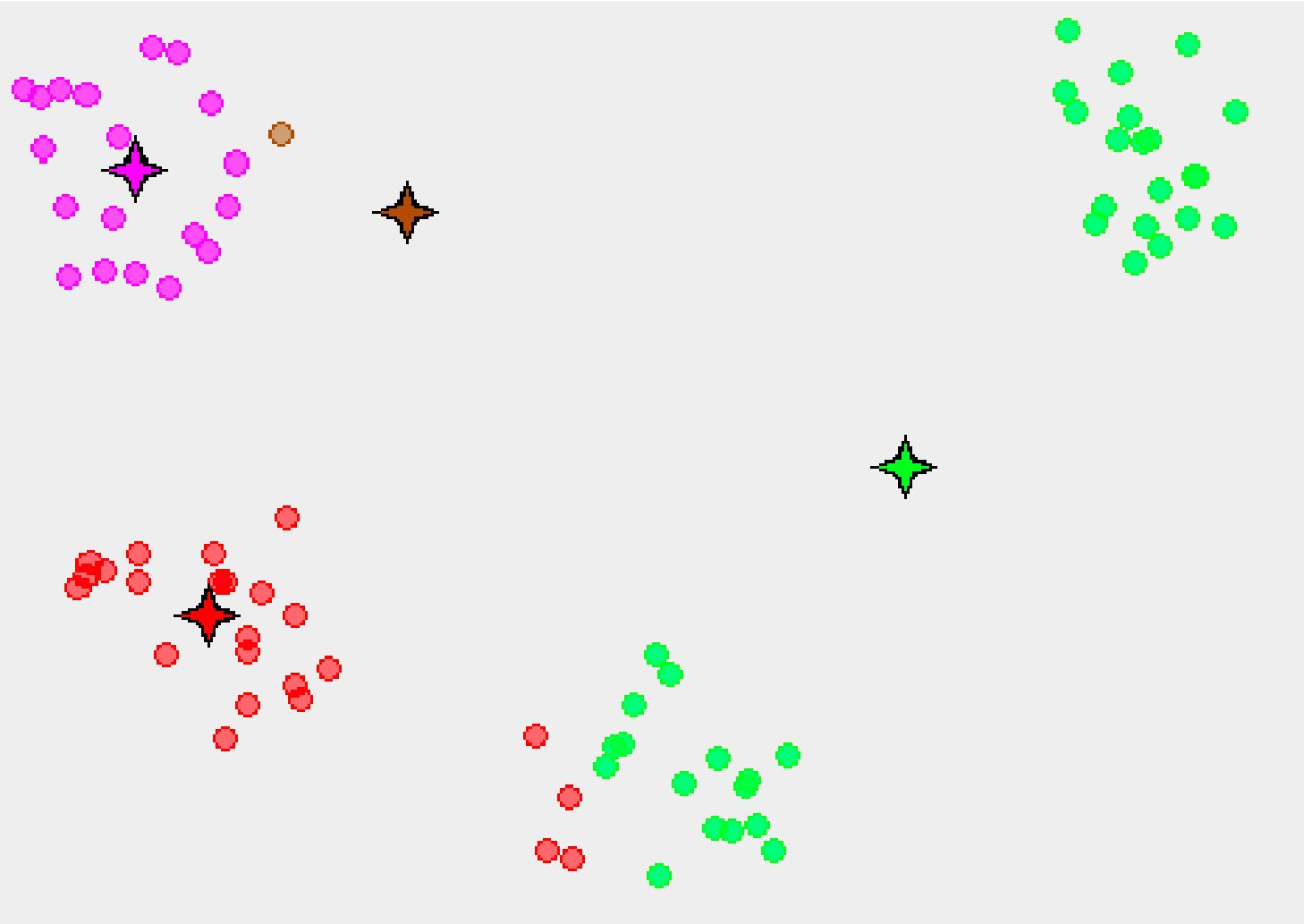
https://commons.wikimedia.org/wiki/File:K-means_convergence_to_a_local_minimum.png

# K-Means Clustering (2)



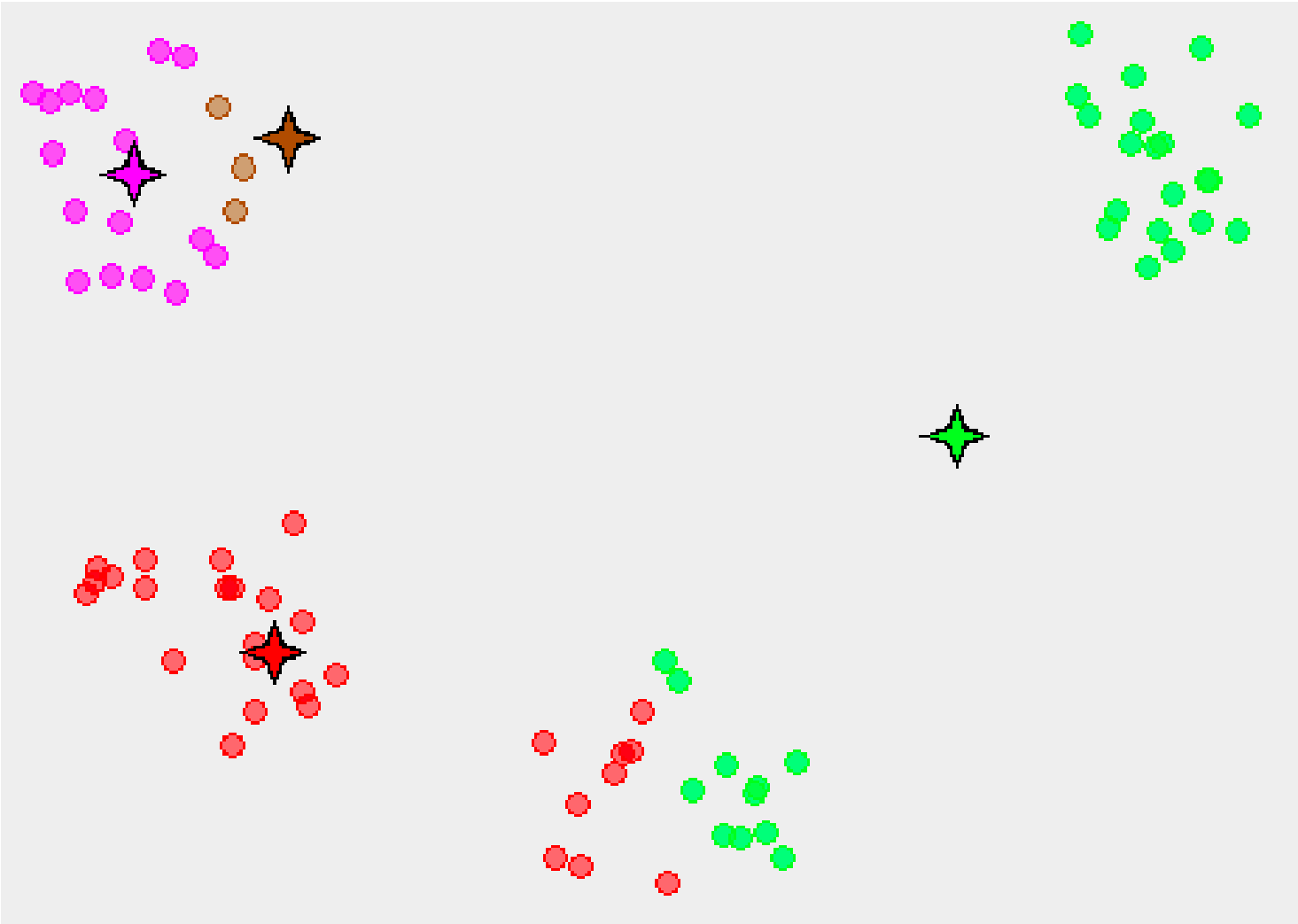
https://commons.wikimedia.org/wiki/File:K-means_convergence_to_a_local_minimum.png

# K-Means Clustering (3)



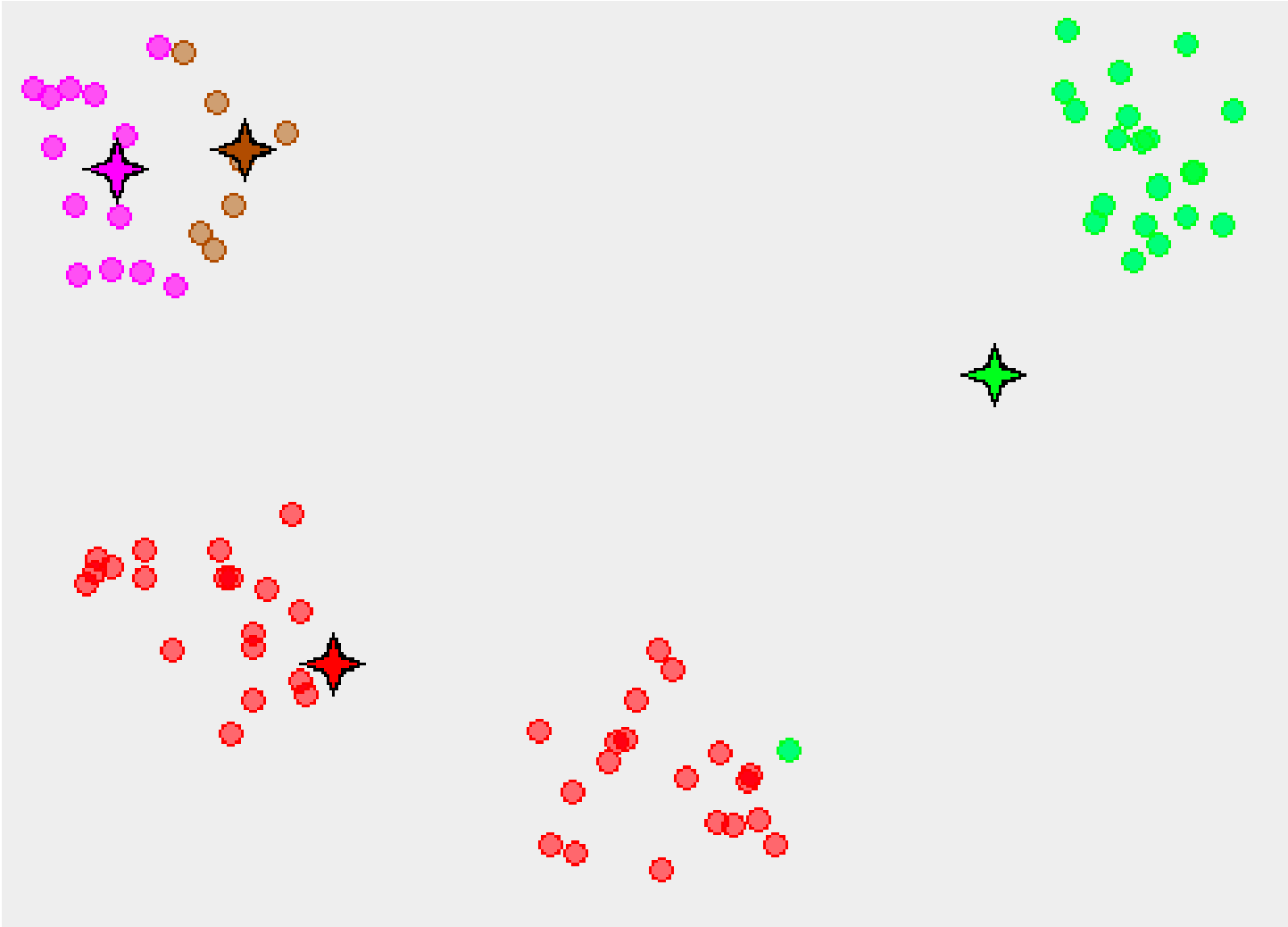
https://commons.wikimedia.org/wiki/File:K-means_convergence_to_a_local_minimum.png

# K-Means Clustering (4)



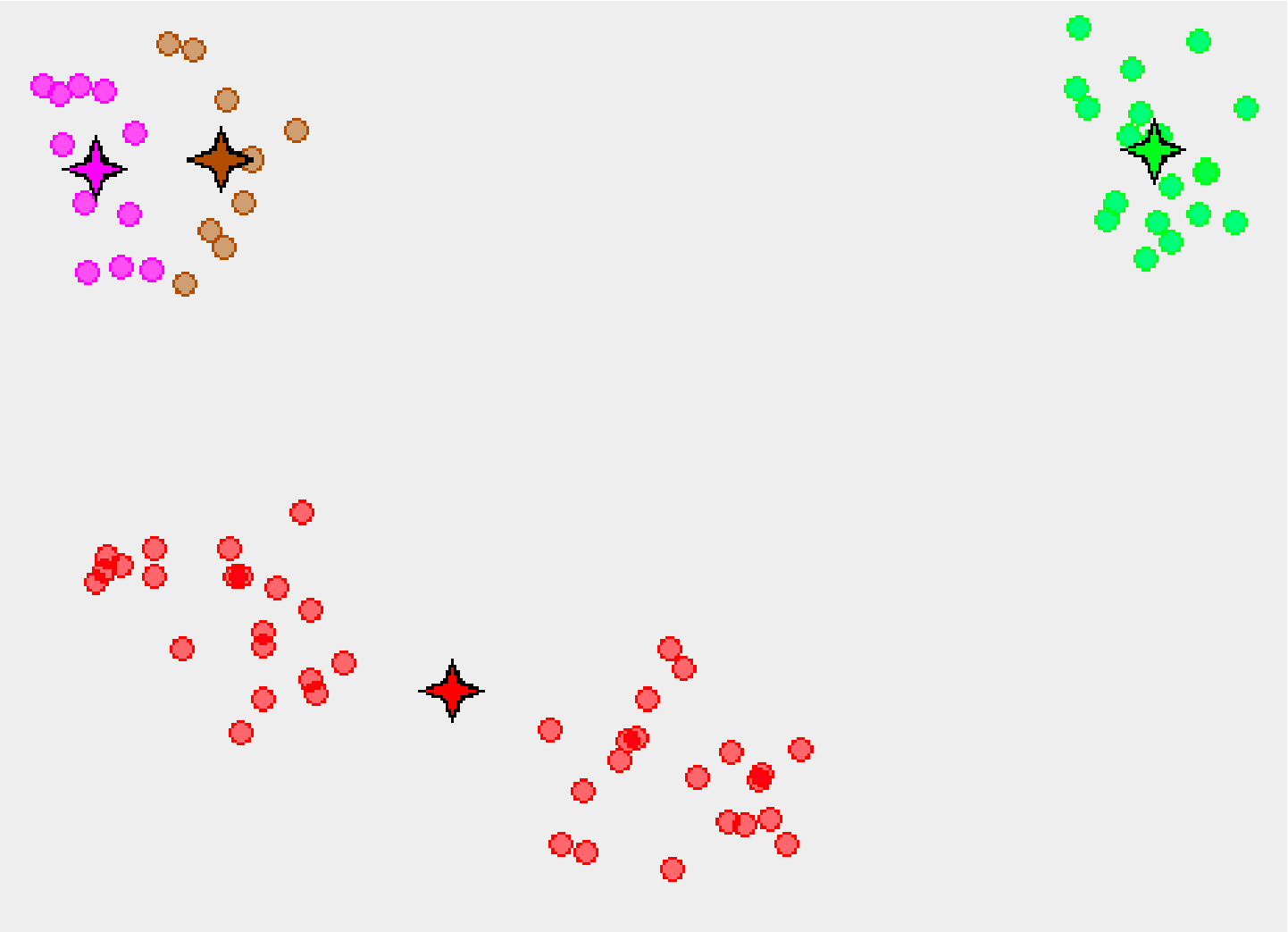
https://commons.wikimedia.org/wiki/File:K-means_convergence_to_a_local_minimum.png

# K-Means Clustering (5)



https://commons.wikimedia.org/wiki/File:K-means_convergence_to_a_local_minimum.png

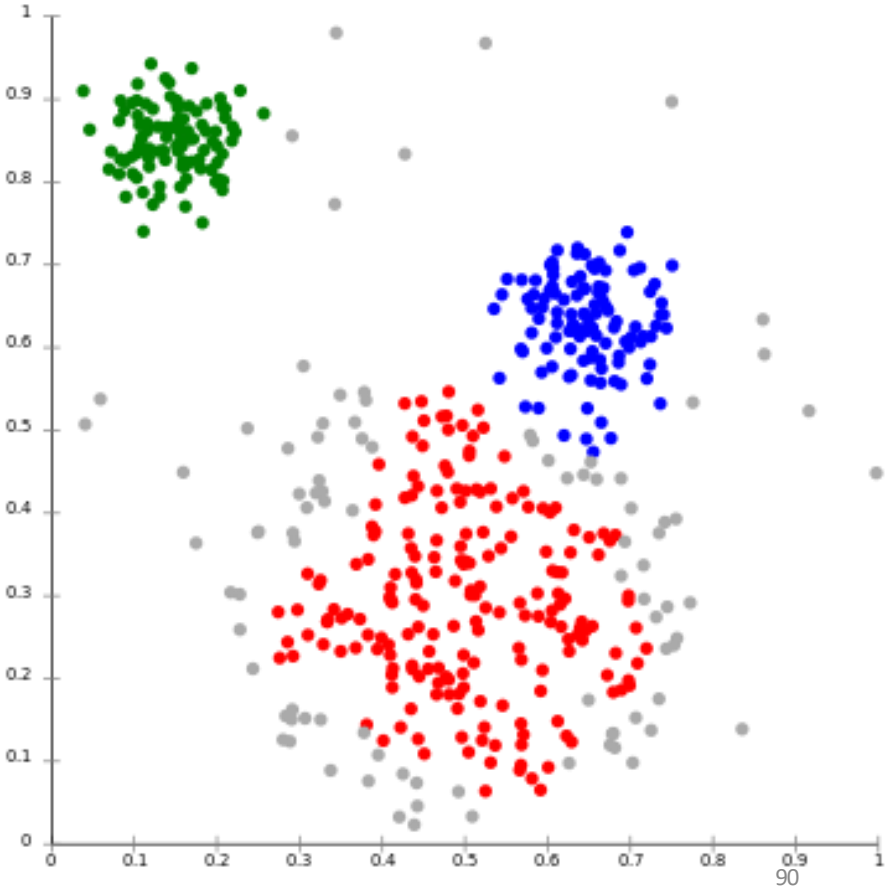
# K-Means Clustering (6)



https://commons.wikimedia.org/wiki/File:K-means_convergence_to_a_local_minimum.png

# Clustering Methods

- Hierarchical clustering
  - Attach datapoints to root points
- K-Means clustering
  - Centroid-based
- Density-based methods
  - Clusters contain a minimal number of datapoints
- …




https://commons.wikimedia.org/wiki/File:DBSCAN-Gaussian-data.svg

# Summary of Concepts

- Supervised learning, unsupervised learning, semi-supervised learning
- Patterns
  - Pattern language
- Streaming data
- Concept drift
- Pattern detection, pattern learning, pattern discovery

- Clustering
  - Feature vectors
- Algorithms:
  - Hierarchical Clustering
    - Dendrogram
  - K-means: cluster centers, centroids